

## Práctica. Frontend en .NET con autenticación.

**Instrucciones.** Elabore una aplicación web de .NET utilizando el patrón de diseño de software MVC con controladores, vistas y modelos que utilice el método de autenticación Cookies y se comunique con un backend API utilizando JWT.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno\_FrontendNetAuth.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
  - a. Coloca las capturas de pantalla de tu código fuente de tus archivos donde se agrega la funcionalidad de la aplicación como controladores, modelos, vistas, middleware y servicios.
  - b. Coloca la captura de pantalla donde se vea la página de inicio de sesión.
  - c. Coloca la captura de pantalla donde se vea la lista de categorías.
  - d. Coloca la captura de pantalla donde se vea la lista de productos.
  - e. Coloca la captura de pantalla donde se vea la lista de usuarios.
  - f. Coloca las capturas de pantalla donde se vea la edición de productos, categorías y usuarios.
  - g. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte de práctica archivo Word a la actividad en Eminus.

## Prerrequisitos

1. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C# y la extensión REST Client.  
*Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).*
2. Esta práctica tiene como prerrequisito tener instalado tener instalado el SDK de .NET descárguelo desde <https://dotnet.microsoft.com/en-us/download/dotnet/8.0>.
3. Esta práctica tiene como prerrequisito tener instalado MySQL en su equipo.  
*Puede seguir las instrucciones de cómo instalar MySQL desde la práctica [Instalación de MySQL](#).*
4. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

## Instrucciones. Defina su Frontend.

Lo primero que tenemos que realizar, es definir las páginas de su frontend. no comience a escribir código y crear estructuras sin saber lo que se quiere realizar. Esta práctica crea la siguiente aplicación:

| URL                                  | Descripción                                                        | Rol requerido          |
|--------------------------------------|--------------------------------------------------------------------|------------------------|
| <i>GET /</i>                         | Inicio de la aplicación                                            | Ninguno                |
| <i>GET /home</i>                     | Inicio de la aplicación                                            | Ninguno                |
| <i>GET /auth</i>                     | Inicio de sesión de usuario                                        | Ninguno                |
| <i>GET /carrito</i>                  | Obtener el carrito de compras                                      | Usuario, Administrador |
| <i>GET /comprar</i>                  | Obtener todas las productos para comprar                           | Usuario, Administrador |
| <i>GET /productos</i>                | Obtener todas las productos                                        | Usuario, Administrador |
| <i>GET /productos?s=título</i>       | Obtener todas las productos que contengan en el título la cadena s | Usuario, Administrador |
| <i>GET /productos/detalle/{id}</i>   | Detalles de una producto por ID                                    | Usuario, Administrador |
| <i>GET /productos/crear</i>          | Agregar una nueva producto                                         | Administrador          |
| <i>GET /productos/editar/{id}</i>    | Actualizar una producto existente                                  | Administrador          |
| <i>GET /productos/eliminar/{id}</i>  | Eliminar una producto                                              | Administrador          |
| <i>GET /categorias</i>               | Obtener todas las categorías                                       | Administrador          |
| <i>GET /categorias/detalle/{id}</i>  | Obtener un categoría por ID                                        | Administrador          |
| <i>GET /categorias/crear</i>         | Agregar una nueva categoría                                        | Administrador          |
| <i>GET /categorias/editar/{id}</i>   | Actualizar una categoría existente                                 | Administrador          |
| <i>GET /categorias/eliminar/{id}</i> | Eliminar una categoría                                             | Administrador          |
| <i>GET /usuarios</i>                 | Obtener todas los usuarios                                         | Administrador          |
| <i>GET /usuarios/detalle/{id}</i>    | Obtener un usuario por ID                                          | Administrador          |
| <i>GET /usuarios/crear</i>           | Agregar un nuevo usuario                                           | Administrador          |
| <i>GET /usuarios/editar/{id}</i>     | Actualizar un usuario existente                                    | Administrador          |
| <i>GET /usuarios/eliminar/{id}</i>   | Eliminar un usuario                                                | Administrador          |

Es momento de comenzar la construcción de la aplicación.

## ¿Qué es la autenticación basada en cookies?

La autenticación basada en cookies ha sido el método predeterminado (y comprobado) para manejar la autenticación de usuarios durante mucho tiempo.

La autenticación basada en cookies presenta un estado (es stateful).

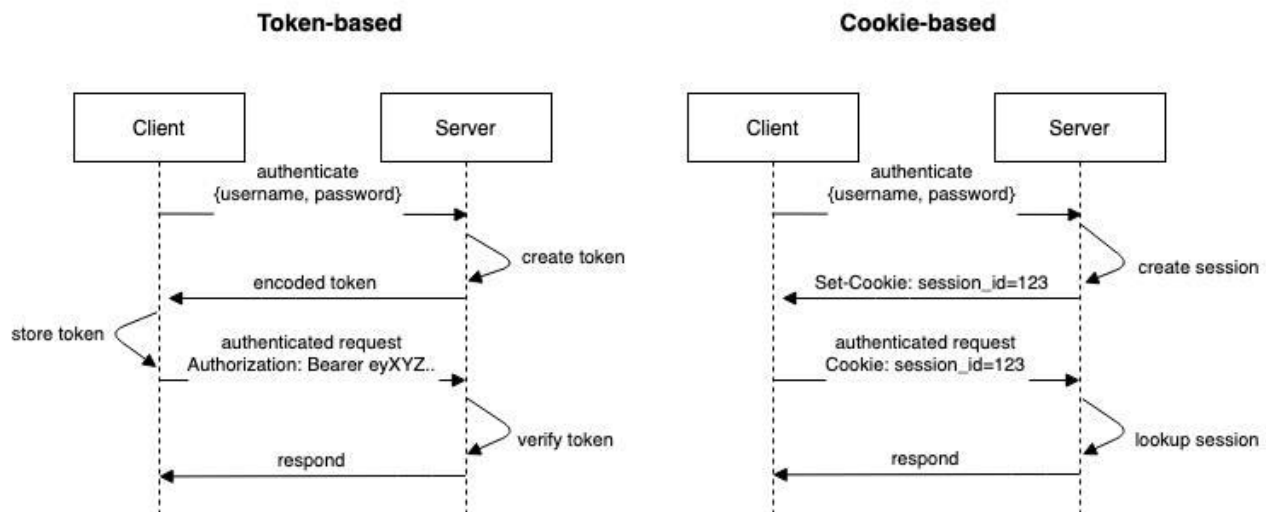
Al iniciar sesión, luego que un usuario envía sus credenciales (y estas se validan), el servidor registra datos (con el fin de recordar que el usuario se ha identificado correctamente). *Estos datos que se registran en el backend*, en correspondencia con el identificador de sesión, es lo que se conoce como *estado*.

En el lado del cliente una cookie es creada para almacenar el identificador de sesión, mientras que los datos se almacenan en el servidor (y son llamados variables de sesión).

## ¿Cómo funciona la autenticación basada en cookies?

El flujo que sigue este sistema de autenticación tradicional es el siguiente:

- Un usuario ingresa sus credenciales (datos que le permiten iniciar sesión).
- El servidor verifica que las credenciales sean correctas, y crea una sesión (esto puede corresponderse con la creación de un archivo, un registro nuevo en una base de datos, o alguna otra solución server-side).
- Una cookie con el **session ID** es puesta en el navegador web del usuario.
- En las peticiones siguientes, el **session ID** es comparado con las sesiones creadas por el servidor.
- Una vez que el usuario se desconecta, la sesión es destruida en ambos lados (tanto en el cliente como en el servidor).



### Instrucciones: Creación de la aplicación.

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
2. Seleccione **Archivo > Abrir carpeta** en el menú principal.
3. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\frontendnet** y selecciónela. Luego haga clic en Seleccionar carpeta.
4. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
5. Abra una Terminal de consola presionando **Ctrl + ñ**.
6. Verifique que tenga instalado el SDK de .NET para crear aplicaciones.

```
dotnet --version  
8.0.303
```

7. Si recibe un mensaje de comando no reconocido, es necesario que descargue e instale el SDK de .NET desde <https://dotnet.microsoft.com/en-us/download/dotnet/8.0>
8. Ejecute el comando siguiente para crear su proyecto web: **dotnet new web**.

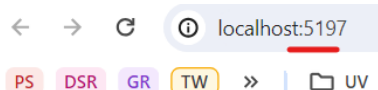
```
PS C:\codigo\tw\frontendnet> dotnet new web  
La plantilla "ASP.NET Core vacío" se creó correctamente.
```

```
Procesando acciones posteriores a la creación...  
Restaurando C:\codigo\tw\frontendnet\frontendnet.csproj:  
  Determinando los proyectos que se van a restaurar...  
  Se ha restaurado C:\codigo\tw\frontendnet\frontendnet.csproj (en 103 ms).  
Restauración realizada correctamente.
```

9. Esto creará un **proyecto web vacío**. Existe también una plantilla disponible que ya genera un proyecto **MVC completo**, pero en este caso, vamos a hacerlo nosotros desde cero.
10. Su web app esta lista para ser ejecutada. En la consola escriba **dotnet build**. Y verifique que no marque errores.
11. Escriba **dotnet run** o **dotnet watch run** para ejecutar su aplicación. La diferencia con ambos, es que la segunda aplica los cambios al guardar el archivo de código fuente modificado sin necesidad de reiniciar todo el servidor web de desarrollo.

```
PS C:\codigo\tw\frontendnet> dotnet run  
Compilando...  
info: Microsoft.Hosting.Lifetime[14]  
      Now listening on: http://localhost:5197  
info: Microsoft.Hosting.Lifetime[0]  
      Application started. Press Ctrl+C to shut down.  
info: Microsoft.Hosting.Lifetime[0]  
      Hosting environment: Development  
info: Microsoft.Hosting.Lifetime[0]  
      Content root path: C:\codigo\tw\frontendnet
```

12. Sino se abre una ventana de su navegador, presione **Ctrl + clic** sobre la dirección **http://localhost:puerto** o copie y péguela para abrirla en Chrome y verifique que su sitio pueda verse.



← → ↻ ⓘ localhost:5197

PS DSR GR TW » | UV

Hello World!



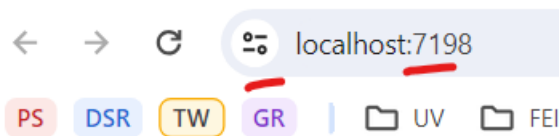
13. Observe como la aplicación se ejecuta en un puerto distinto al estándar 80 (HTTP) o al 443 (HTTPS).
14. En caso de que quisiéramos ejecutar la aplicación con HTTPS, escriba `dotnet dev-certs https --trust` para confiar en el certificado de desarrollo HTTPS local.



15. En caso de que se le solicite, acepte el certificado SSL auto firmado.
16. Para utilizar SSL, utilice `dotnet run --launch-profile https` o `dotnet watch run --launch-profile https` según corresponda.

```
PS C:\codigo\tw\mvc> dotnet run --launch-profile https
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7198
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5101
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\codigo\tw\mvc
```

17. Observe como se sirve el contenido con un certificado SSL.



Hello World!

18. Para terminar la ejecución escriba en la consola `Ctrl + C` en Visual Studio Code.
19. Para cambiar el puerto en el que se ejecutará su aplicación, abra el archivo `/Properties/launchSettings.json`.
20. Los perfiles disponibles cuando usted ejecuta su aplicación en el servidor de desarrollo son `http`, `https` y si lo tuviera instalado en Windows, `IIS Express`.
21. Cambie el puerto del perfil default `http` por `8080`.

```
"profiles": {
  "http": {
    "commandName": "Project",
    "dotnetRunMessages": true,
    "launchBrowser": true,
    "applicationUrl": "http://localhost:8080",
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  },
}
```

22. Ejecute su aplicación usando `dotnet run` o `dotnet watch run` y ahora puede acceder a la ventana de su navegador en el nuevo puerto.

```
PS C:\codigo\tw\frontendnet> dotnet run
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:8080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
```

23. Felicidades, ha creado su aplicación de manera correcta.

### Instrucciones: Configuración inicial.

1. Primeramente vamos a obtener una lista de librerías de cliente necesarias para el proyecto. Se obtendrán desde redes de distribución de contenidos y se muestran a continuación.

| Librería de cliente           | URL                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jquery                        | <a href="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.1/jquery.min.js">https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.1/jquery.min.js</a>                                                                                                                                                                                                            |
| jquery-validate               | <a href="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.20.0/jquery.validate.min.js">https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.20.0/jquery.validate.min.js</a>                                                                                                                                                                      |
| jquery-validation-unobtrusive | <a href="https://cdnjs.cloudflare.com/ajax/libs/jquery-validation-unobtrusive/4.0.0/jquery.validate.unobtrusive.min.js">https://cdnjs.cloudflare.com/ajax/libs/jquery-validation-unobtrusive/4.0.0/jquery.validate.unobtrusive.min.js</a>                                                                                                                    |
| bootstrap                     | <a href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.3/js/bootstrap.bundle.min.js">https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.3/js/bootstrap.bundle.min.js</a><br><a href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.3/css/bootstrap.min.css">https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.3/css/bootstrap.min.css</a> |
| bootstrap-icons               | <a href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css</a>                                                                                                                                                                              |

2. En la barra de herramientas superior del **Explorador de archivos**, seleccione **Nueva carpeta** y cree las siguientes carpetas:
  - **Controllers**. Aquí van los controladores.
  - **Middelwares**. Aquí van los middlewares.
  - **Models**. Aquí almacenaremos los modelos.
  - **Services**. Aquí van los servicios.
  - **Views**. Aquí van las vistas. Debemos crear subcarpetas con el nombre de cada controlador.
  - **Views/Archivos**. Aquí van las vistas para el controlador Archivos.
  - **Views/Auth**. Aquí van las vistas para el controlador Auth.
  - **Views/Bitacora**. Aquí van las vistas para el controlador Bitacora.
  - **Views/Carrito**. Aquí van las vistas para el controlador Carrito.
  - **Views/Categorias**. Aquí van las vistas para el controlador Categorías.
  - **Views/Comprar**. Aquí van las vistas para el controlador Comprar.
  - **Views/Home**. Aquí van las vistas para el controlador Home.
  - **Views/Productos**. Aquí van las vistas para el controlador Productos.
  - **Views/Perfil**. Aquí van las vistas para el controlador Perfil.
  - **Views/Shared**. Aquí van las vistas compartidas.
  - **Views/Usuarios**. Aquí van las vistas para el controlador Usuarios.
  - **wwwroot**. Aquí va el contenido estático.
  - **wwwroot/css/**. Aquí van las hojas de estilo.
  - **wwwroot/images/**. Aquí van las imágenes.
  - **wwwroot/js/**. Aquí van los scripts js.
3. Descargue [el código](#) para cambiar el tema del sitio y colóquelo en la carpeta **wwwroot/js/cambio\_tema.js**.
4. Descargue [el código](#) para cambiar el tema del sitio y colóquelo en la carpeta **wwwroot/js/portada.js**.
5. Descargue [el código](#) para cambiar el tema del sitio y colóquelo en la carpeta **wwwroot/js/producto\_portada.js**.
6. Descargue el logo de su aplicación que se verá en el menú del sitio desde [netlogo.jpg](#) y colóquelo en la carpeta **wwwroot/images/logo.png**.
7. Descargue el logo de su aplicación que se verá en el menú del sitio desde [logo.png](#) y colóquelo en la carpeta **wwwroot/images/logo.png**.
8. Descargue el logo de su aplicación que se verá en el menú del sitio desde [logo-dark.png](#) y colóquelo en la carpeta **wwwroot/images/logo-dark.png**.

9. Descargue los cuatro fondos del carrusel de su aplicación que se verá en la pantalla principal del sitio desde [fondos](#) y colóquelo en la carpeta `wwwroot/images/`.
10. Descargue el icono de su aplicación que se ve en la pestaña del navegador desde [favicon.svg](#) y colóquelo en la carpeta `wwwroot/favicon.svg`.
11. En la carpeta `wwwroot/css/` agregue un nuevo archivo llamado `site.css` con el siguiente código.

```
html[data-bs-theme="dark"] body {
  --mercadolibre-bg: var(--bs-body-bg);
  --logo: url(../images/logo-dark.png) no-repeat;
}

html[data-bs-theme="light"] body {
  --mercadolibre-bg: #ffe600;
  --logo: url(../images/logo.png) no-repeat;
}

.bg-mercadolibre {
  background-color: var(--mercadolibre-bg) !important;
}

.logo {
  display: block;
  -moz-box-sizing: border-box;
  box-sizing: border-box;
  background: var(--logo);
  background-size: 100px 25px;
  height: 25px;
  width: 100px;
}
```

12. Abra su archivo `appsettings.json` y agregue la conexión hacia su backend API. En este ejemplo es la siguiente, pero puede variar en su equipo.

```
},
"AllowedHosts": "*",
"URLWebAPI": "http://localhost:3000"
}
```

13. Enhorabuena. Ha configurado correctamente su aplicación.

### Instrucciones. Agregar los modelos con validación de entrada de usuario.

En .NET para realizar la validación de entrada del usuario, se puede utilizar los atributos de validación de modelos por medio de la clase `System.ComponentModel.DataAnnotations`. Esto ofrece la funcionalidad de validar la entrada tanto desde código de dinámico como desde la **Vista** utilizando **JavaScript** por medio de librería `jquery-validate` que forma parte del paquete que incluye .NET en sus plantillas para frontend.

1. En la carpeta **Models** agregue un nuevo archivo llamado **AuthUser.cs** con el siguiente código.

```
namespace frontendnet.Models;

public class AuthUser
{
    public required string Email { get; set; }
    public required string Nombre { get; set; }
    public required string Rol { get; set; }
    public required string Jwt { get; set; }
}
```

2. En la carpeta **Models** agregue un nuevo archivo llamado **Categoria.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Categoria
{
    [Display(Name = "Id")]
    public int? CategoriaId { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public required string Nombre { get; set; }

    [Display(Name = "Eliminable")]
    public bool Protegida { get; set; } = false;
}
```

3. En la carpeta **Models** agregue un nuevo archivo llamado **Login.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Login
{
    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [EmailAddress(ErrorMessage = "El campo {0} no es correo válido.")]
    [Display(Name = "Correo electrónico")]
    public required string Email { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [DataType(DataType.Password)]
    [Display(Name = "Contraseña")]
    public required string Password { get; set; }
}
```

- Observe los atributos de validación como `Required` y `EmailAddress`. Observe como utilizaremos este modelo mostrar el inicio de sesión en la vista correspondiente.
- En la carpeta `Models` agregue un nuevo archivo llamado `Producto.cs` con el siguiente código

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet.Models;

public class Producto
{
    [Display(Name = "Id")]
    public int? ProductoId { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public required string Titulo { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [DataType(DataType.MultilineText)]
    public string Descripcion { get; set; } = "Sin descripción";

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [DataType(DataType.Currency)]
    [RegularExpression(@"^\d+.\d{0,2}$", ErrorMessage = "El valor del campo debe ser un precio válido.")]
    [DisplayFormat(DataFormatString = "{0:C}", ApplyFormatInEditMode = false)]
    [Display(Name = "Precio")]
    public decimal Precio { get; set; }

    [Display(Name = "Portada")]
    public int? ArchivoId { get; set; }

    [Display(Name = "Eliminable")]
    public bool Protegida { get; set; } = false;

    public ICollection<Categoria>? Categorias { get; set; }
}
```

- En la carpeta `Models` agregue un nuevo archivo llamado `ProductoCategoria.cs` con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class ProductoCategoria
{
    [Display(Name = "Categoría")]
    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public int? CategoriaId { get; set; }

    public string? Nombre { get; set; }

    public Producto? Producto { get; set; }
}
```

7. En la carpeta **Models** agregue un nuevo archivo llamado **Rol.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Rol
{
    public required string Id { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [Display(Name = "Rol")]
    public required string Nombre { get; set; }
}
```

8. En la carpeta **Models** agregue un nuevo archivo llamado **Usuario.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Usuario
{
    [Display(Name = "Id")]
    public string? Id { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [EmailAddress(ErrorMessage = "El campo {0} no es correo válido.")]
    public required string Email { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public required string Nombre { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public required string Rol { get; set; }
}
```

9. En la carpeta **Models** agregue un nuevo archivo llamado **UsuarioPwd.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class UsuarioPwd
{
    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [EmailAddress(ErrorMessage = "El campo {0} no es correo válido.")]
    [Display(Name = "Correo electrónico")]
    public required string Email { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [MinLength(6, ErrorMessage = "El campo {0} debe tener un mínimo de {1} caracteres.")]
    [DataType(DataType.Password)]
    [Display(Name = "Contraseña")]
    public required string Password { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public required string Nombre { get; set; }

    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    public required string Rol { get; set; }
}
```

10. En la carpeta **Models** agregue un nuevo archivo llamado **Upload.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Upload
{
    [Display(Name = "Id")]
    public int? ArchivoId { get; set; }

    public string? Nombre { get; set; }

    [Display(Name = "Portada")]
    [Required(ErrorMessage = "El campo {0} es obligatorio.")]
    [DataType(DataType.Upload)]
    public required IFormFile Portada { get; set; }
}
```

11. En la carpeta **Models** agregue un nuevo archivo llamado **Archivo.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Archivo
{
    [Display(Name = "Id")]
    public int? ArchivoId { get; set; }

    [Display(Name = "MIME")]
    public string? Mime { get; set; }

    public string? Nombre { get; set; }

    [Display(Name = "Tamprecio")]
    public int? Size { get; set; }

    [Display(Name = "Repositorio")]
    public bool InDb { get; set; } = true;
}
```



12. En la carpeta **Models** agregue un nuevo archivo llamado **Bitacora.cs** con el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace frontendnet.Models;

public class Bitacora
{
    [Display(Name = "Id")]
    public int? BitacoraId { get; set; }

    [Display(Name = "Acción")]
    public string? Accion { get; set; }

    [Display(Name = "Elemento Id")]
    public string? ElementoId { get; set; }

    [Display(Name = "IP")]
    public string? IP { get; set; }

    [Display(Name = "Usuario")]
    public string? Usuario { get; set; }

    [Display(Name = "Fecha")]
    public DateTime? Fecha { get; set; }
}
```

13. Enhorabuena, ha creado correctamente este apartado.

### Instrucciones. Agregar middlewares.

Los middlewares son funciones de código que se ejecutan antes de que una petición HTTP llegue al manejador de rutas o antes de que un cliente reciba una respuesta, lo que da al framework la capacidad de ejecutar un script típico antes o después de la petición de un cliente.

1. En la carpeta **Middlewares** agregue un nuevo archivo llamado **EnviaBearerDelegatingHandler.cs** con el siguiente código.

```
using System.Security.Claims;

namespace frontendnet.Middlewares;

public class EnviaBearerDelegatingHandler(IHttpContextAccessor httpContextAccessor) : DelegatingHandler
{
    protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken cancellationToken)
    {
        request.Headers.Add("Authorization", "Bearer " + httpContextAccessor.HttpContext?.User.FindFirstValue("jwt"));
        return base.SendAsync(request, cancellationToken);
    }
}
```

2. En la carpeta **Middlewares** agregue un nuevo archivo llamado **RefrescaTokenDelegatingHandler.cs** con el siguiente código.

```
using System.Security.Claims;
using frontendnet.Services;

namespace frontendnet.Middlewares;

public class RefrescaTokenDelegatingHandler(AuthClientService auth, IHttpContextAccessor httpContextAccessor) : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken cancellationToken)
    {
        var response = await base.SendAsync(request, cancellationToken);
        response.EnsureSuccessStatusCode();
        // Revisa si el servidor nos envió un nuevo token
        if (response.Headers.Contains("Set-Authorization"))
        {
            string jwt = response.Headers.GetValues("Set-Authorization").FirstOrDefault();
            var claims = new List<Claim>
            {
                // Todo esto se guarda en la Cookie
                new(ClaimTypes.Name, httpContextAccessor.HttpContext?.User.FindFirstValue(ClaimTypes.Name)!),
                new(ClaimTypes.GivenName, httpContextAccessor.HttpContext?.User.FindFirstValue(ClaimTypes.GivenName)!),
                new("jwt", jwt),
                new(ClaimTypes.Role, httpContextAccessor.HttpContext?.User.FindFirstValue(ClaimTypes.Role)!),
            };
            auth.IniciaSesionAsync(claims);
        }
        return response;
    }
}
```

3. Enhorabuena, ha creado correctamente este apartado.

## Instrucciones. Agregar los Servicios.

1. Agregue un nuevo archivo llamado **AuthClientService.cs** en la carpeta **Services** con el siguiente código.

```
using frontendnet.Models;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;

namespace frontendnet.Services;

public class AuthClientService(HttpClient client, IHttpContextAccessor httpContextAccessor)
{
    public async Task<AuthUser> ObtenTokenAsync(string email, string password)
    {
        Login usuario = new() { Email = email, Password = password };
        // Realizo la llamada al Web API
        var response = await client.PostAsJsonAsync("api/auth", usuario);
        var token = await response.Content.ReadFromJsonAsync<AuthUser>();

        return token!;
    }

    public async void IniciaSesionAsync(List<Claim> claims)
    {
        var claimsIdentity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
        var authProperties = new AuthenticationProperties();
        await httpContextAccessor.HttpContext?.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, new ClaimsPrincipal(claimsIdentity), authProperties)!;
    }
}
```

2. Agregue un nuevo archivo llamado **CategoriasClientService.cs** en la carpeta **Services** con el siguiente código.

```
using frontendnet.Models;

namespace frontendnet.Services;

public class CategoriasClientService(HttpClient client)
{
    public async Task<List<Categoria>>> GetAsync()
    {
        return await client.GetFromJsonAsync<List<Categoria>>>("api/categorias");
    }

    public async Task<Categoria>> GetAsync(int id)
    {
        return await client.GetFromJsonAsync<Categoria>($"api/categorias/{id}");
    }

    public async Task PostAsync(Categoria categoria)
    {
        var response = await client.PostAsJsonAsync($"api/categorias", categoria);
        response.EnsureSuccessStatusCode();
    }

    public async Task PutAsync(Categoria categoria)
    {
        var response = await client.PutAsJsonAsync($"api/categorias/{categoria.CategoriaId}", categoria);
        response.EnsureSuccessStatusCode();
    }

    public async Task DeleteAsync(int id)
    {
        var response = await client.DeleteAsync($"api/categorias/{id}");
        response.EnsureSuccessStatusCode();
    }
}
```

3. Agregue un nuevo archivo llamado **ProductosClientService.cs** en la carpeta **Services** con el siguiente código.

```
using frontendnet.Models;

namespace frontendnet.Services;

public class ProductosClientService(HttpClient client)
{
    public async Task<List<Producto>>> GetAsync(string? search)
    {
        return await client.GetFromJsonAsync<List<Producto>>>($"api/productos?s={search}");
    }

    public async Task<Producto>> GetAsync(int id)
    {
        return await client.GetFromJsonAsync<Producto>($"api/productos/{id}");
    }

    public async Task PostAsync(Producto producto)
    {
        var response = await client.PostAsJsonAsync($"api/productos", producto);
        response.EnsureSuccessStatusCode();
    }

    public async Task PutAsync(Producto producto)
    {
        var response = await client.PutAsJsonAsync($"api/productos/{producto.ProductoId}", producto);
        response.EnsureSuccessStatusCode();
    }

    public async Task DeleteAsync(int id)
    {
        var response = await client.DeleteAsync($"api/productos/{id}");
        response.EnsureSuccessStatusCode();
    }

    public async Task PostAsync(int id, int categoriaid)
    {
        var response = await client.PostAsJsonAsync($"api/productos/{id}/categoria", new { categoriaid });
        response.EnsureSuccessStatusCode();
    }

    public async Task DeleteAsync(int id, int categoriaid)
    {
        var response = await client.DeleteAsync($"api/productos/{id}/categoria/{categoriaid}");
        response.EnsureSuccessStatusCode();
    }
}
```

4. Agregue un nuevo archivo llamado **PerfilClientService.cs** en la carpeta **Services** con el siguiente código.

```
namespace frontendnet.Services;

public class PerfilClientService(HttpClient client)
{
    public async Task<string> ObtenTiempoAsync()
    {
        return await client.GetStringAsync($"api/auth/tiempo");
    }
}
```

5. Agregue un nuevo archivo llamado **RolesClientService.cs** en la carpeta **Services** con el siguiente código.

```
using frontendnet.Models;

namespace frontendnet.Services;

public class RolesClientService(HttpClient client)
{
    public async Task<List<Rol>?> GetAsync()
    {
        return await client.GetFromJsonAsync<List<Rol>>("api/roles");
    }
}
```

6. Agregue un nuevo archivo llamado **UsuariosClientService.cs** en la carpeta **Services** con el siguiente código.

```
using frontendnet.Models;

namespace frontendnet.Services;

public class UsuariosClientService(HttpClient client)
{
    public async Task<List<Usuario>?> GetAsync()
    {
        return await client.GetFromJsonAsync<List<Usuario>>("api/usuarios");
    }

    public async Task<Usuario>? GetAsync(string email)
    {
        return await client.GetFromJsonAsync<Usuario>($"api/usuarios/{email}");
    }

    public async Task PostAsync(UsuarioPwd usuario)
    {
        var response = await client.PostAsJsonAsync($"api/usuarios", usuario);
        response.EnsureSuccessStatusCode();
    }

    public async Task PutAsync(Usuario usuario)
    {
        var response = await client.PutAsJsonAsync($"api/usuarios/{usuario.Email}", usuario);
        response.EnsureSuccessStatusCode();
    }

    public async Task DeleteAsync(string email)
    {
        var response = await client.DeleteAsync($"api/usuarios/{email}");
        response.EnsureSuccessStatusCode();
    }
}
```

7. Agregue un nuevo archivo llamado **ArchivosClientService.cs** en la carpeta **Services** con el siguiente código.

```
using System.Net.Http.Headers;
using frontendnet.Models;

namespace frontendnet.Services;

public class ArchivosClientService(HttpClient client)
{
    public async Task<List<Archivo>>> GetAsync()
    {
        return await client.GetFromJsonAsync<List<Archivo>>>("api/archivos");
    }

    public async Task<Archivo>> GetAsync(int id)
    {
        return await client.GetFromJsonAsync<Archivo>>($"api/archivos/{id}/detalle");
    }

    public async Task PostAsync(Upload Archivo)
    {
        var memoryStream = new MemoryStream();
        await Archivo.Portada.CopyToAsync(memoryStream);
        var Contenido = memoryStream.ToArray();
        memoryStream.Close();
        var fileContent = new ByteArrayContent(Contenido);
        fileContent.Headers.ContentType = MediaTypeHeaderValue.Parse(Archivo.Portada.ContentType);
        using var form = new MultipartFormDataContent
        {
            { fileContent, "file", Archivo.Portada.FileName! }
        };

        var response = await client.PostAsync($"api/archivos", form);
        response.EnsureSuccessStatusCode();
    }

    public async Task PutAsync(Upload Archivo)
    {
        var memoryStream = new MemoryStream();
        await Archivo.Portada.CopyToAsync(memoryStream);
        var Contenido = memoryStream.ToArray();
        memoryStream.Close();
        var fileContent = new ByteArrayContent(Contenido);
        fileContent.Headers.ContentType = MediaTypeHeaderValue.Parse(Archivo.Portada.ContentType);
        using var form = new MultipartFormDataContent
        {
            { fileContent, "file", Archivo.Portada.FileName! }
        };

        var response = await client.PutAsync($"api/archivos/{Archivo.ArchivoId}", form);
        response.EnsureSuccessStatusCode();
    }

    public async Task DeleteAsync(int id)
    {
        var response = await client.DeleteAsync($"api/archivos/{id}");
        response.EnsureSuccessStatusCode();
    }
}
```

8. Agregue un nuevo archivo llamado `BitacoraClientService.cs` en la carpeta `Services` con el siguiente código.

```
using frontendnet.Models;

namespace frontendnet.Services;

public class BitacoraClientService(HttpClient client)
{
    public async Task<List<Bitacora>?> GetAsync()
    {
        return await client.GetFromJsonAsync<List<Bitacora>>("api/bitacora");
    }
}
```

9. Enhorabuena, ha creado correctamente este apartado.

## Instrucciones. Agregar los Controladores.

### Controlador: Archivos

1. Agregue un nuevo archivo llamado `ArchivosController.cs` en la carpeta `Controllers` con el siguiente código.

```
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

[Authorize(Roles = "Administrador")]
public class ArchivosController(ArchivosClientService archivos, IConfiguration configuration) : Controller
{
    public async Task<IActionResult> Index()
    {
        List<Archivo>? lista = [];
        try
        {
            lista = await archivos.GetAsync();
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
        ViewBag.Url = configuration["UrlWebAPI"];
        return View(lista);
    }
}
```

2. Dentro de la clase agregue los siguientes métodos.

```
public async Task<IActionResult> Detalle(int id)
{
    Archivo? item = null;
    try
    {
        item = await archivos.GetAsync(id);
        if (item == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.Url = configuration["UrlWebAPI"];
    return View(item);
}
```



```
public IActionResult Crear()
{
    return View();
}

[HttpPost]
public async Task<IActionResult> CrearAsync(Upload itemToCreate)
{
    ViewBag.Url = configuration["UrlWebAPI"];
    if (ModelState.IsValid)
    {
        try
        {
            {
                if ((itemToCreate.Portada.Length / 1024) > 100)
                {
                    ModelState.AddModelError("Portada", $"El archivo de {itemToCreate.Portada.Length / 1024} KB supera el tamprecio máximo permitido.");
                    return View(itemToCreate);
                }
                if (itemToCreate.Portada.ContentType != "image/jpeg")
                {
                    ModelState.AddModelError("Portada", $"El archivo {itemToCreate.Portada.FileName} no tiene una extensión permitida.");
                    return View(itemToCreate);
                }

                await archivos.PostAsync(itemToCreate);
                return RedirectToAction(nameof(Index));
            }
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            {
                return RedirectToAction("Salir", "Auth");
            }
        }

        ModelState.AddModelError("Portada", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
        return View(itemToCreate);
    }
}

public async Task<IActionResult> EditarAsync(int id)
{
    ViewBag.Url = configuration["UrlWebAPI"];
    try
    {
        {
            Archivo? itemToEdit = await archivos.GetAsync(id);
            ViewBag.ArchivoId = itemToEdit?.ArchivoId;
            ViewBag.Nombre = itemToEdit?.Nombre;
            if (itemToEdit == null) return NotFound();
        }
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
        {
            return RedirectToAction("Salir", "Auth");
        }
    }

    return View();
}

[HttpPost]
public async Task<IActionResult> EditarAsync(int id, Upload itemToEdit)
{
    if (itemToEdit == null) return NotFound();
    if (id != itemToEdit.ArchivoId) return NotFound();

    ViewBag.ArchivoId = itemToEdit.ArchivoId;
    ViewBag.Nombre = itemToEdit.Nombre;

    ViewBag.Url = configuration["UrlWebAPI"];
    if (ModelState.IsValid)
    {
        try
        {
            {
                if ((itemToEdit.Portada.Length / 1024) > 100)
                {
                    ModelState.AddModelError("Portada", $"El archivo de {itemToEdit.Portada.Length / 1024} KB supera el tamprecio máximo permitido.");
                    return View(itemToEdit);
                }
                if (itemToEdit.Portada.ContentType != "image/jpeg")
                {
                    ModelState.AddModelError("Portada", $"El archivo {itemToEdit.Portada.FileName} no tiene una extensión permitida.");
                    return View(itemToEdit);
                }

                await archivos.PutAsync(itemToEdit);
                return RedirectToAction(nameof(Index));
            }
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            {
                return RedirectToAction("Salir", "Auth");
            }
        }

        ModelState.AddModelError("Portada", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
        return View(itemToEdit);
    }
}
```

```
public async Task<IActionResult> Eliminar(int id, bool? showError = false)
{
    Archivo? itemToDelete = null;
    try
    {
        itemToDelete = await archivos.GetAsync(id);
        if (itemToDelete == null) return NotFound();

        if (showError.GetValueOrDefault())
            ViewData["ErrorMessage"] = "No ha sido posible realizar la acción. Inténtelo nuevamente.";
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.Url = configuration["UrlWebAPI"];
    return View(itemToDelete);
}

[HttpPost]
public async Task<IActionResult> Eliminar(int id)
{
    ViewBag.Url = configuration["UrlWebAPI"];
    if (ModelState.IsValid)
    {
        try
        {
            await archivos.DeleteAsync(id);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    return RedirectToAction(nameof(Eliminar), new { id, showError = true });
}
```

## Controlador: Auth

3. Agregue un nuevo archivo llamado **AuthController.cs** en la carpeta **Controllers** con el siguiente código.

```
using System.Security.Claims;
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

public class AuthController(AuthClientService auth) : Controller
{
    [AllowAnonymous]
    public IActionResult Index()
    {
        return View();
    }
}
```

4. Dentro de la clase agregue los siguientes métodos.

```
[HttpPost]
[AllowAnonymous]
[ValidateAntiForgeryToken]
public async Task<IActionResult> IndexAsync(Login model)
{
    if (ModelState.IsValid)
    {
        try
        {
            // Esta función verifica en backend que el correo y contraseña sean válidos
            var token = await auth.ObtenTokenAsync(model.Email, model.Password);
            var claims = new List<Claim>
            {
                // Todo esto se guarda en la Cookie
                new(ClaimTypes.Name, token.Email),
                new(ClaimTypes.GivenName, token.Nombre),
                new("jwt", token.Jwt),
                new(ClaimTypes.Role, token.Rol),
            };
            auth.IniciaSesionAsync(claims);
            // Usuario válido
            if (token.Rol == "Administrador")
                return RedirectToAction("Index", "Productos");
            else
                return RedirectToAction("Index", "Home");
        }
        catch (Exception)
        {
            ModelState.AddModelError("Email", "Credenciales no válidas. Inténtelo nuevamente.");
        }
    }
    return View(model);
}

[Authorize(Roles = "Administrador, Usuario")]
public async Task<IActionResult> SalirAsync()
{
    // Cierra la sesión
    await HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
    // Sino, se redirige a la página inicial
    return RedirectToAction("Index", "Auth");
}
```

### Controlador: Bitácora

5. Agregue un nuevo archivo llamado `BitacoraController.cs` en la carpeta `Controllers` con el siguiente código.

```
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

[Authorize(Roles = "Administrador")]
public class BitacoraController(BitacoraClientService bitacora) : Controller
{
    public async Task<IActionResult> IndexAsync()
    {
        List<Bitacora>? lista = [];
        try
        {
            lista = await bitacora.GetAsync();
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
        return View(lista);
    }
}
```

### Controlador: Categorías

6. Agregue un nuevo archivo llamado **CategoriasController.cs** en la carpeta **Controllers** con el siguiente código.

```
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

[Authorize(Roles = "Administrador")]
public class CategoriasController(CategoriasClientService categorias) : Controller
{
    public async Task<IActionResult> Index()
    {
        List<Categoria>? lista = [];
        try
        {
            lista = await categorias.GetAsync();
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
        return View(lista);
    }
}
```

7. Dentro de la clase agregue los siguientes métodos.

```
public async Task<IActionResult> Detalle(int id)
{
    Categoria? item = null;
    try
    {
        item = await categorias.GetAsync(id);
        if (item == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    return View(item);
}

public IActionResult Crear()
{
    return View();
}

[HttpPost]
public async Task<IActionResult> CrearAsync(Categoria itemToCreate)
{
    if (ModelState.IsValid)
    {
        try
        {
            await categorias.PostAsync(itemToCreate);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }

    ModelState.AddModelError("Nombre", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    return View(itemToCreate);
}

public async Task<IActionResult> EditarAsync(int id)
{
    Categoria? itemToEdit = null;
    try
    {
        itemToEdit = await categorias.GetAsync(id);
        if (itemToEdit == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    return View(itemToEdit);
}

[HttpPost]
public async Task<IActionResult> EditarAsync(int id, Categoria itemToEdit)
{
    if (id != itemToEdit.CategoriaId) return NotFound();

    if (ModelState.IsValid)
    {
        try
        {
            await categorias.PutAsync(itemToEdit);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }

    ModelState.AddModelError("Nombre", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    return View(itemToEdit);
}
```

```
public async Task<IActionResult> Eliminar(int id, bool? showError = false)
{
    Categoria? itemToDelete = null;
    try
    {
        itemToDelete = await categorias.GetAsync(id);
        if (itemToDelete == null) return NotFound();

        if (showError.GetValueOrDefault())
            ViewData["ErrorMessage"] = "No ha sido posible realizar la acción. Inténtelo nuevamente.";
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    return View(itemToDelete);
}

[HttpPost]
public async Task<IActionResult> Eliminar(int id)
{
    if (ModelState.IsValid)
    {
        try
        {
            await categorias.DeleteAsync(id);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    return RedirectToAction(nameof(Eliminar), new { id, showError = true });
}
```

## Controlador: Home

8. Agregue un nuevo archivo llamado **HomeController.cs** en la carpeta **Controllers** con el siguiente código.

```
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }

    public IActionResult Error([FromServices] IHostEnvironment hostEnvironment)
    {
        return View();
    }

    public IActionResult AccessDenied()
    {
        return View();
    }
}
```

## Controlador: Productos

9. Agregue un nuevo archivo llamado `ProductosController.cs` en la carpeta `Controllers` con el siguiente código.

```
using System.Security.Claims;
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace frontendnet;

[Authorize(Roles = "Administrador")]
public class ProductosController(ProductosClientService productos,
    CategoriasClientService categorias,
    ArchivosClientService archivos,
    IConfiguration configuration) : Controller
{
    public async Task<IActionResult> Index(string? s)
    {
        List<Producto>? lista = [];
        try
        {
            lista = await productos.GetAsync(s);
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
        if (User.FindFirstValue(ClaimTypes.Role) == "Administrador")
            ViewBag.SoloAdmin = true;

        ViewBag.Url = configuration["UrlWebAPI"];
        ViewBag.search = s;
        return View(lista);
    }
}
```

10. Dentro de la clase agregue los siguientes métodos.

```
public async Task<IActionResult> Detalle(int id)
{
    Producto? item = null;
    ViewBag.Url = configuration["UrlWebAPI"];
    try
    {
        item = await productos.GetAsync(id);
        if (item == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    return View(item);
}
```

```
public async Task<IActionResult> Crear()
{
    await ProductosDropDownListAsync();
    return View();
}

[HttpPost]
public async Task<IActionResult> CrearAsync(Producto itemToCreate)
{
    ViewBag.Url = configuration["UrlWebAPI"];
    if (ModelState.IsValid)
    {
        try
        {
            await productos.PostAsync(itemToCreate);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }

    await ProductosDropDownListAsync();
    ModelState.AddModelError("Nombre", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    return View(itemToCreate);
}

public async Task<IActionResult> EditarAsync(int id)
{
    Producto? itemToEdit = null;
    ViewBag.Url = configuration["UrlWebAPI"];
    try
    {
        itemToEdit = await productos.GetAsync(id);
        if (itemToEdit == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    await ProductosDropDownListAsync();
    return View(itemToEdit);
}

[HttpPost]
public async Task<IActionResult> EditarAsync(int id, Producto itemToEdit)
{
    if (id != itemToEdit.ProductoId) return NotFound();

    ViewBag.Url = configuration["UrlWebAPI"];
    if (ModelState.IsValid)
    {
        try
        {
            await productos.PutAsync(itemToEdit);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    await ProductosDropDownListAsync();
    ModelState.AddModelError("Nombre", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    return View(itemToEdit);
}
```



```
public async Task<IActionResult> Eliminar(int id, bool? showError = false)
{
    Producto? itemToDelete = null;
    try
    {
        itemToDelete = await productos.GetAsync(id);
        if (itemToDelete == null) return NotFound();

        if (showError.GetValueOrDefault())
            ViewData["ErrorMessage"] = "No ha sido posible realizar la acción. Inténtelo nuevamente.";
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.Url = configuration["UrlWebAPI"];
    return View(itemToDelete);
}

[HttpPost]
public async Task<IActionResult> Eliminar(int id)
{
    ViewBag.Url = configuration["UrlWebAPI"];
    if (ModelState.IsValid)
    {
        try
        {
            await productos.DeleteAsync(id);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    // En caso de error
    return RedirectToAction(nameof(Eliminar), new { id, showError = true });
}

[AcceptVerbs("GET", "POST")]
public IActionResult ValidaPoster(string Poster)
{
    if (Uri.IsWellFormedUriString(Poster, UriKind.Absolute) || Poster.Equals("N/A"))
        return Json(true);
    return Json(false);
}

public async Task<IActionResult> Categorias(int id)
{
    Producto? itemToView = null;
    try
    {
        itemToView = await productos.GetAsync(id);
        if (itemToView == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewData["ProductoId"] = itemToView?.ProductoId;
    ViewBag.Url = configuration["UrlWebAPI"];
    return View(itemToView);
}
```

```
public async Task<IActionResult> CategoriasAgregar(int id)
{
    ProductoCategoria? itemToView = null;
    try
    {
        Producto? producto = await productos.GetAsync(id);
        if (producto == null) return NotFound();

        await CategoriasDropDownListAsync();
        itemToView = new ProductoCategoria { Producto = producto };
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.Url = configuration["UrlWebAPI"];
    return View(itemToView);
}

[HttpPost]
public async Task<IActionResult> CategoriasAgregar(int id, int categoriaid)
{
    Producto? producto = null;
    if (ModelState.IsValid)
    {
        try
        {
            producto = await productos.GetAsync(id);
            if (producto == null) return NotFound();

            Categoria? categoria = await categorias.GetAsync(categoriaid);
            if (categoria == null) return NotFound();

            await productos.PostAsync(id, categoriaid);
            return RedirectToAction(nameof(Categorias), new { id });
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    // En caso de error
    ViewBag.Url = configuration["UrlWebAPI"];
    ModelState.AddModelError("id", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    await CategoriasDropDownListAsync();
    return View(new ProductoCategoria { Producto = producto });
}
```

```
public async Task<IActionResult> CategoriasRemover(int id, int categoriaid, bool? showError = false)
{
    ProductoCategoria? itemToView = null;
    try
    {
        Producto? producto = await productos.GetAsync(id);
        if (producto == null) return NotFound();

        Categoria? categoria = await categorias.GetAsync(categoriaid);
        if (categoria == null) return NotFound();

        itemToView = new ProductoCategoria { Producto = producto, CategoriaId = categoriaid, Nombre = categoria.Nombre };

        if (showError.GetValueOrDefault())
            ViewData["ErrorMessage"] = "No ha sido posible realizar la acción. Inténtelo nuevamente.";
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.Url = configuration["UrlWebAPI"];
    return View(itemToView);
}

[HttpPost]
public async Task<IActionResult> CategoriasRemover(int id, int categoriaid)
{
    if (ModelState.IsValid)
    {
        try
        {
            await productos.DeleteAsync(id, categoriaid);
            return RedirectToAction(nameof(Categorias), new { id });
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    // En caso de error
    return RedirectToAction(nameof(CategoriasRemover), new { id, categoriaid, showError = true });
}

private async Task CategoriasDropDownListAsync(object? itemSeleccionado = null)
{
    var listado = await categorias.GetAsync();
    ViewBag.Categoria = new SelectList(listado, "CategoriaId", "Nombre", itemSeleccionado);
}

private async Task ProductosDropDownListAsync(object? itemSeleccionado = null)
{
    var listado = await archivos.GetAsync();
    ViewBag.Archivo = new SelectList(listado, "ArchivoId", "Nombre", itemSeleccionado);
}
```

## Controlador: Perfil

11. Agregue un nuevo archivo llamado `PerfilController.cs` en la carpeta `Controllers` con el siguiente código.

```
using System.Security.Claims;
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

[Authorize]
public class PerfilController(PerfilClientService perfil) : Controller
{
    public async Task<IActionResult> IndexAsync()
    {
        AuthUser? usuario = null;
        try
        {
            ViewBag.timeRemaining = await perfil.ObtenTiempoAsync();

            // Obtiene el perfil del usuario
            usuario = new AuthUser
            {
                Email = User.FindFirstValue(ClaimTypes.Name)!,
                Nombre = User.FindFirstValue(ClaimTypes.GivenName)!,
                Rol = User.FindFirstValue(ClaimTypes.Role)!,
                Jwt = User.FindFirstValue("jwt")!
            };
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
        return View(usuario);
    }
}
```

### Controlador: Usuarios

12. Agregue un nuevo archivo llamado **UsuariosController.cs** en la carpeta **Controllers** con el siguiente código.

```
using System.Security.Claims;
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace frontendnet;

[Authorize(Roles = "Administrador")]
public class UsuariosController(UsuariosClientService usuarios, RolesClientService roles) : Controller
{
    public async Task<IActionResult> Index()
    {
        List<Usuario>? lista = [];
        try
        {
            lista = await usuarios.GetAsync();
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
        return View(lista);
    }
}
```

13. Dentro de la clase agregue los siguientes métodos.

```
public async Task<IActionResult> Detalle(string id)
{
    Usuario? item = null;
    try
    {
        item = await usuarios.GetAsync(id);
        if (item == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    return View(item);
}
```

```
public async Task<IActionResult> Crear()
{
    await RolesDropDownListAsync();
    return View();
}

[HttpPost]
public async Task<IActionResult> CrearAsync(UsuarioPwd itemToCreate)
{
    if (ModelState.IsValid)
    {
        try
        {
            await usuarios.PostAsync(itemToCreate);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }

    ModelState.AddModelError("Email", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    await RolesDropDownListAsync();
    return View(itemToCreate);
}

[HttpGet("[controller]/[action]/{email}")]
public async Task<IActionResult> EditarAsync(string email)
{
    Usuario? itemToEdit = null;
    try
    {
        itemToEdit = await usuarios.GetAsync(email);
        if (itemToEdit == null) return NotFound();
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.PuedeEditar = !(User.Identity?.Name == email);
    await RolesDropDownListAsync(itemToEdit?.Rol);
    return View(itemToEdit);
}

[HttpPost("[controller]/[action]/{email}")]
public async Task<IActionResult> EditarAsync(string email, Usuario itemToEdit)
{
    if (email != itemToEdit.Email) return NotFound();

    if (ModelState.IsValid)
    {
        try
        {
            await usuarios.PutAsync(itemToEdit);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }

    ModelState.AddModelError("Nombre", "No ha sido posible realizar la acción. Inténtelo nuevamente.");
    ViewBag.PuedeEditar = !(User.Identity?.Name == email);
    await RolesDropDownListAsync(itemToEdit?.Rol);
    return View(itemToEdit);
}
```

```
public async Task<IActionResult> Eliminar(string id, bool? showError = false)
{
    Usuario? itemToDelete = null;
    try
    {
        itemToDelete = await usuarios.GetAsync(id);
        if (itemToDelete == null) return NotFound();

        if (showError.GetValueOrDefault())
            ViewData["ErrorMessage"] = "No ha sido posible realizar la acción. Inténtelo nuevamente.";
    }
    catch (HttpRequestException ex)
    {
        if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            return RedirectToAction("Salir", "Auth");
    }
    ViewBag.PuedeEditar = !(User.Identity?.Name == id);
    return View(itemToDelete);
}

[HttpPost]
public async Task<IActionResult> Eliminar(string id)
{
    if (ModelState.IsValid)
    {
        try
        {
            await usuarios.DeleteAsync(id);
            return RedirectToAction(nameof(Index));
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
                return RedirectToAction("Salir", "Auth");
        }
    }
    return RedirectToAction(nameof(Eliminar), new { id, showError = true });
}

private async Task RolesDropDownListAsync(object? rolSeleccionado = null)
{
    var listado = await roles.GetAsync();
    ViewBag.Rol = new SelectList(listado, "Nombre", "Nombre", rolSeleccionado);
}
```

### Controlador: Carrito

14. Agregue un nuevo archivo llamado **UsuariosController.cs** en la carpeta **Controllers** con el siguiente código.

```
using System.Security.Claims;
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace frontendnet;

[Authorize(Roles = "Usuario")]
public class CarritoController() : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

### Controlador: Comprar

15. Agregue un nuevo archivo llamado **UsuariosController.cs** en la carpeta **Controllers** con el siguiente código.

```
using frontendnet.Models;
using frontendnet.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace frontendnet;

[Authorize(Roles = "Usuario")]
public class ComprarController(ProductosClientService productos, IConfiguration configuration) : Controller
{
    public async Task<IActionResult> Index(string? s)
    {
        List<Producto>? lista = [];
        try
        {
            lista = await productos.GetAsync(s);
        }
        catch (HttpRequestException ex)
        {
            if (ex.StatusCode == System.Net.HttpStatusCode.Unauthorized)
            {
                return RedirectToAction("Salir", "Auth");
            }
        }

        ViewBag.Url = configuration["UrlWebAPI"];
        ViewBag.search = s;
        return View(lista);
    }
}
```

16. Enhorabuena, ha creado correctamente este apartado.



Instrucciones: Crear la plantilla de frontend en el motor de plantillas.

1. Generalmente se debe crear una o más plantillas de diseño para ser utilizadas por varias páginas web, en lugar de definir el diseño en cada una de las páginas web.
2. El motor de vistas de frontend incluido con .NET se llama **Razor View Engine**. Debe configurarlo para poder darle toda la funcionalidad a sus vistas.

### Carpeta: Views

3. Agregue un nuevo archivo llamado **ViewImports.cshtml** en la carpeta **Views** con el siguiente contenido.

```
@using frontendnet
@using frontendnet.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

4. Este es un archivo especial de Razor Engine y siempre se incluirá en las vistas.
5. Agregue un nuevo archivo llamado **ViewStart.cshtml** en la carpeta **Views** con el siguiente contenido.

```
@{
    Layout = "_Layout";
}
```

6. Este es un archivo especial de Razor Engine y siempre se incluirá en las vistas.

### Carpeta: Views/Shared

7. Agregue un nuevo archivo llamado **ValidationScriptsPartial.cshtml** en la carpeta **Views/Shared** con el siguiente contenido.

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.20.0/jquery.validate.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validation-unobtrusive/4.0.0/jquery.validate.unobtrusive.min.js"></script>
<script>
    var settings = {
        validClass: "is-valid",
        errorClass: "is-invalid"
    };
    $.validator.setDefaults(settings);
    $.validator.unobtrusive.options = settings;
</script>
```

8. Observe que son las URL que habíamos obtenido previamente. Este solo se debe incluir en las vistas que requieran validación de entradas.
9. Agregue un nuevo archivo llamado **Error.cshtml** en la carpeta **Views/Shared** con el siguiente contenido.

```
@{
    ViewData["Title"] = "Aplicación no disponible";
}

<div class="container">
    <h1 class="text-danger">@ViewData["Title"]</h1>
    <p class="text-muted">La aplicación se encuentra no disponible. Intentelo nuevamente más tarde.</p>
</div>
```

10. Agregue un nuevo archivo llamado `AccessDenied.cshtml` en la carpeta `Views/Shared` con el siguiente contenido.

```
@{
    ViewData["Title"] = "Acceso denegado";
}

<div class="container">
    <h1 class="text-danger">@ViewData["Title"]</h1>
    <p class="text-muted">El usuario actual no tiene acceso a este apartado.</p>
</div>
```

11. Agregue un nuevo archivo llamado `MenuDerPartial.cshtml` en la carpeta `Views/Shared` con el siguiente contenido.

```
<div class="navbar-nav small">
    <div class="navbar-text form-check form-switch form-check-reverse me-2">
        <input class="form-check-input" type="checkbox" id="darkModeSwitch">
        <label class="form-check-label" for="darkModeSwitch"><i class="bi bi-moon-stars modo-luz"></i></label>
    </div>
    @if (User.Identity?.IsAuthenticated == true)
    {
        <a class="nav-link" asp-controller="Perfil">Hola @(User.Identity.Name)!</a>
        <a class="nav-link" asp-controller="Auth" asp-action="Salir">Salir</a>
    }
    else
    {
        <a class="nav-link" asp-controller="Auth" asp-action="Index">Iniciar sesión</a>
    }
    <div class="navbar-text">
        
    </div>
</div>
```

12. Este archivo es una vista parcial del menú principal del sitio.
13. Agregue un nuevo archivo llamado `MenuIzqPartial.cshtml` en la carpeta `Views/Shared` con el siguiente contenido.

```
@using System.Security.Claims

<div class="me-auto navbar-nav">

    @if (User.Identity?.IsAuthenticated == true)
    {
        @if (User.FindFirstValue(ClaimTypes.Role) == "Usuario")
        {
            <a class="nav-link" asp-controller="Comprar" asp-action="Index">Comprar</a>
            <a class="nav-link" asp-controller="Carrito" asp-action="Index"><i class="bi bi-cart"></i> Carrito</a>
        }
        else if (User.FindFirstValue(ClaimTypes.Role) == "Administrador")
        {
            <a class="nav-link" asp-controller="Productos" asp-action="Index">Productos</a>
            <a class="nav-link" asp-controller="Categorias" asp-action="Index">Categorias</a>
            <a class="nav-link" asp-controller="Archivos" asp-action="Index">Archivos</a>
            <a class="nav-link" asp-controller="Usuarios" asp-action="Index">Usuarios</a>
            <a class="nav-link" asp-controller="Bitacora" asp-action="Index">Bitácora</a>
        }
    }
    else
    {
        <a class="nav-link" asp-controller="Home" asp-action="Index">Inicio</a>
    }
</div>
```

14. Este archivo también es una vista parcial del menú principal del sitio.
15. Agregue un nuevo archivo llamado **Layout.cshtml** en la carpeta **Views/Shared** con el siguiente contenido.

```
<!DOCTYPE html>
<html lang="es" data-bs-theme="light">

<head>
    <meta charset="utf-8" />
    <link rel="icon" href="~/favicon.svg">
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Mercado Libre Fei">
    <title>Mercado Libre: @ViewData["Title"]</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.3/css/bootstrap.min.css" />
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
    <link rel="stylesheet" href="~/css/site.css" />
</head>

<body class="bg-body-tertiary">
    <header>
        <nav class="navbar navbar-expand-md bg-mercadolibre">
            <div class="container-fluid">
                <a class="navbar-brand" asp-controller="Home" asp-action="Index">
                    <span class="logo" alt="Inicio" height="25"></span>
                </a>
                <button class="navbar-toggler shadow-none border-0 btn btn-light" type="button" data-bs-toggle="collapse"
                    data-bs-target=".navbar-collapse" aria-controls="navbarSupportedContent" aria-expanded="false"
                    aria-label="Toggle navigation">
                    <i class="bi bi-three-dots"></i>
                </button>
                <div class="navbar-collapse collapse">
                    <partial name="_MenuIzqPartial" />
                    <partial name="_MenuDerPartial" />
                </div>
            </div>
        </nav>
    </header>
    <main role="main" class="mt-3 mb-5 container">
        @RenderBody()
    </main>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.3/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/cambio_tema.js"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>

</html>
```

16. Enhorabuena, su plantilla está lista para ser utilizada.

## Instrucciones. Agregar las vistas

### Vista: Archivos

1. Dentro de la carpeta llamada `Views/Archivos` agregue un nuevo archivo llamado `DetallePartial.cshtml` con el siguiente código.

```
@model Archivo

<div class="card mt-3">
    <div class="row g-0">
        <div class="col-md-4">
            
        </div>
        <div class="col-md-8">
            <div class="card-body">
                <div class="mb-3 ps-3">
                    <label asp-for="ArchivoId" class="form-label form-text"></label>
                    <div class="form-text text-body">@Model.ArchivoId</div>
                </div>
                <div class="mb-3 ps-3">
                    <label asp-for="Nombre" class="form-label form-text"></label>
                    <div class="form-text text-body">@Model.Nombre</div>
                </div>
                <div class="mb-3 ps-3">
                    <label asp-for="Mime" class="form-label form-text"></label>
                    <div class="form-text text-body">@Model.Mime</div>
                </div>
                <div class="mb-3 ps-3">
                    <label asp-for="Size" class="form-label form-text"></label>
                    <div class="form-text text-body">@((Model.Size/1024) KB</div>
                </div>
                <div class="mb-3 ps-3">
                    <label asp-for="InDb" class="form-label form-text"></label>
                    <div class="form-text text-body">@((Model.InDb ? "Base de datos" : "Archivos"))</div>
                </div>
            </div>
        </div>
    </div>
</div>
```

2. Dentro de la carpeta llamada **Views/Archivos** agregue un nuevo archivo llamado **Crear.cshtml** con el siguiente código.

```
@model Upload
@{
    ViewData["Title"] = "Archivos";
    ViewData["SubTitle"] = "Crear";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script src="~/js/portada.js"></script>
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Crear" enctype="multipart/form-data">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="row g-0">
            <div class="col-md-4">
                
            </div>
            <div class="col-md-8">
                <div class="card-body small">
                    <div class="form-floating mb-3">
                        <input asp-for="Portada" accept=".jpg" placeholder="Titulo" class="form-control" />
                        <label asp-for="Portada" class="form-label"></label>
                        <span asp-validation-for="Portada" class="text-danger"></span>
                        <div class="form-text small">Tamprecio máximo permitido es 100 KB</div>
                        <div class="form-text small">Las extensiones permitidas son (jpg)</div>
                    </div>
                </div>
            </div>
        </div>
        <div class="text-center mt-3">
            <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </div>
</form>
```

3. Dentro de la carpeta llamada `Views/Archivos` agregue un nuevo archivo llamado `Detalle.cshtml` con el siguiente código.

```
@model Archivo
@{
    ViewData["Title"] = "Archivos";
    ViewData["SubTitle"] = "Detalle";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<partial name="_DetallePartial" model="Model" />
<div class="text-center mt-3">
    <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
</div>
```

4. Dentro de la carpeta llamada **Views/Archivos** agregue un nuevo archivo llamado **Editar.cshtml** con el siguiente código.

```
@model Upload
@{
    ViewData["Title"] = "Archivos";
    ViewData["SubTitle"] = "Editar";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script src="~/js/portada.js"></script>
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Editar" enctype="multipart/form-data">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="row g-0">
            <div class="col-md-4">
                
            </div>
            <div class="col-md-8">
                <div class="card-body small">
                    <div class="form-floating mb-3">
                        <input asp-for="ArchivoId" placeholder="ArchivoId" readonly class="form-control-plaintext"
                            value="@ViewBag.ArchivoId" />
                        <label asp-for="ArchivoId" class="form-label"></label>
                    </div>
                    <div class="form-floating mb-3">
                        <input asp-for="Portada" accept=".jpg" placeholder="Titulo" class="form-control" />
                        <label asp-for="Portada" class="form-label"></label>
                        <span asp-validation-for="Portada" class="text-danger"></span>
                        <div class="form-text small">Tamprecio máximo permitido es 100 KB</div>
                        <div class="form-text small">Las extensiones permitidas son (jpg)</div>
                    </div>
                </div>
            </div>
        </div>
        <div class="text-center mt-3">
            <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </div>
</form>
```

5. Dentro de la carpeta llamada **Views/Archivos** agregue un nuevo archivo llamado **Eliminar.cshtml** con el siguiente código.

```
@model Archivo
@{
    ViewData["Title"] = "Archivos";
    ViewData["SubTitle"] = "Eliminar";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<p class="text-danger">¿Está seguro que desea eliminar este elemento?</p>
<p class="text-danger">@ViewData["ErrorMessage"]</p>

<partial name="_DetallePartial" />
<form asp-action="Eliminar">
    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Eliminar</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
</form>
```



6. Dentro de la carpeta llamada **Views/Archivos** agregue un nuevo archivo llamado **Index.cshtml** con el siguiente código.

```
@model List<Archivo>
@{
    ViewData["Title"] = "Archivos";
    ViewData["SubTitle"] = "Listado";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<div class="row small mb-3">
    <div class="col">
        <a class="text-decoration-none" asp-action="Crear" title="Crear nuevo">Agregar nuevo</a>
    </div>
    <div class="col text-end">
        Mostrando @Model.Count() elementos
    </div>
</div>

@if (Model.Count() > 0)
{
    <div class="table-responsive">
        <table class="table table-striped table-bordered small">
            <thead class="text-center">
                <tr>
                    <th>
                        @Html.DisplayNameFor(model => model.First().ArchivoId)
                    </th>
                    <th>Poster</th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Nombre)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Mime)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Size)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().InDb)
                    </th>
                    <th></th>
                    <th></th>
                    <th></th>
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td>
                            @Html.DisplayFor(modelItem => item.ArchivoId)
                        </td>
                        <td class="text-center width="1">
                            
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Nombre)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Mime)
                        </td>
                        <td>
                            @(item.Size / 1024) KB
                        </td>
                        <td>
                            @(item.InDb ? "Base de datos" : "Archivos")
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Detalle"
                                asp-route-id="@item.ArchivoId">Detalle</a>
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Editar"
                                asp-route-id="@item.ArchivoId">Editar</a>
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Eliminar"
                                asp-route-id="@item.ArchivoId">Eliminar</a>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            No se han encontrado elementos. Inténtelo de nuevo más tarde.
        </div>
    </div>
}
}
```

## Vista: Auth

7. Dentro de la carpeta llamada `Views/Auth` agregue un nuevo archivo llamado `Index.cshtml` con el siguiente código.

```
@model Login
@{
    ViewData["Title"] = "Inicio de sesión";
}

<form asp-action="Index">
    <div class="card mx-auto" style="max-width: 500px;">
        <div class="card-body">
            <h1 class="h3 text-center">Bienvenido a Mercado Libre</h1>
            <h2 class="h6 text-center text-muted">Facultad de Estadística e Informática</h2>
            <h5 class="card-card-title text-center mt-4">@ViewData["Title"]</h5>
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-floating mb-3">
                <input asp-for="Email" class="form-control" placeholder="Email">
                <label asp-for="Email" class="form-label"></label>
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-floating mb-3">
                <input asp-for="Password" class="form-control" placeholder="Password">
                <label asp-for="Password" class="form-label"></label>
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <p class="card-text small">Ingrese sus credenciales para poder disfrutar de todo el contenido de la plataforma.</p>
            <p class="card-text small text-end"><a asp-controller="Home" asp-action="Index" class="link-primary text-decoration-none">Crear cuenta</a></p>
        </div>
        <div class="card-footer text-center">
            <button type="submit" class="btn btn-primary">Iniciar sesión</button>
        </div>
    </div>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

## Vista: Bitácora

8. Dentro de la carpeta llamada `Views/Bitacora` agregue un nuevo archivo llamado `Index.cshtml` con el siguiente código.

```
@model List<Bitacora>
@{
    ViewData["Title"] = "Bitácora";
    ViewData["SubTitle"] = "Listado";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<div class="row small mb-3">
    <div class="col text-end">
        Mostrando @Model.Count() elementos
    </div>
</div>

@if (Model.Count() > 0)
{
    <div class="table-responsive">
        <table class="table table-striped table-bordered small">
            <thead class="text-center">
                <tr>
                    <th>
                        @Html.DisplayNameFor(model => model.First().BitacoraId)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Accion)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().ElementoId)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().IP)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Usuario)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Fecha)
                    </th>
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td>
                            @Html.DisplayFor(modelItem => item.BitacoraId)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Accion)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.ElementoId)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.IP)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Usuario)
                        </td>
                        <td>
                            @TimeZoneInfo.ConvertTimeBySystemTimeZoneId(Convert.ToDateTime(item.Fecha), "America/Mexico_City")
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            No se han encontrado elementos. Inténtelo de nuevo más tarde.
        </div>
    </div>
}
}
```

## Vista: Carrito

9. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `_CardPartial.cshtml` con el siguiente código.

```
@model Producto

<div class="card h-100">
    
    <div class="card-body d-flex flex-column">
        <p class="card-text">@Model.Titulo</p>
        <h5 class="card-title">@Html.DisplayFor(model => model.Precio)</h5>
        <p class="card-text">
            @foreach (var cat in Model.Categorias!)
            {
                <span class="badge rounded-pill text-bg-warning">@cat.Nombre</span>
            }
        </p>
        <p class="mt-auto card-text text-success fw-semibold small"><i class="bi bi-lightning-fill"></i> Envío gratis</p>
    </div>
    <div class="card-footer">
        <a class="btn btn-primary btn-sm" href="#" role="button">Comprar</a>
    </div>
</div>
```

10. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `Index.cshtml` con el siguiente código.

```
@{
    ViewData["Title"] = "Carrito de compras";
    ViewData["SubTitle"] = "Listado";
}

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<div class="row small mb-3">
    <div class="col text-end">
        Mostrando 0 elementos
    </div>
</div>

<div class="mt-5">
    <div class="alert alert-warning" role="alert">
        No se han encontrado elementos. Inténtelo de nuevo más tarde.
    </div>
</div>
```

## Vista: Categorías

11. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `DetallePartial.cshtml` con el siguiente código.

```
@model Categoria

<div class="card mt-3">
  <div class="card-body">
    <div class="mb-3 ps-3">
      <label asp-for="CategoriaId" class="form-label form-text"></label>
      <div class="form-text text-body">@Model.CategoriaId</div>
    </div>
    <div class="mb-3 ps-3">
      <label asp-for="Nombre" class="form-label form-text"></label>
      <div class="form-text text-body">@Model.Nombre</div>
    </div>
  </div>
</div>
```

12. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `Crear.cshtml` con el siguiente código.

```
@model Categoria
@{
    ViewData["Title"] = "Categorias";
    ViewData["SubTitle"] = "Crear";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Crear">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="card-body small">
            <div class="form-floating mb-3">
                <input asp-for="Nombre" placeholder="Nombre" class="form-control" />
                <label asp-for="Nombre" class="form-label"></label>
                <span asp-validation-for="Nombre" class="text-danger"></span>
            </div>
        </div>
    </div>

    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
</form>
```

13. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `Detalle.cshtml` con el siguiente código.

```
@model Categoria
@{
    ViewData["Title"] = "Categorias";
    ViewData["SubTitle"] = "Detalle";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<partial name="_DetallePartial" />
<div class="text-center mt-3">
    <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
</div>
```

14. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `Editar.cshtml` con el siguiente código.

```
@model Categoria
@{
    ViewData["Title"] = "Categorias";
    ViewData["SubTitle"] = "Editar";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Editar">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="card-body small">
            <div class="form-floating mb-3">
                <input asp-for="CategoriaId" placeholder="CategoriaId" readonly class="form-control-plaintext" />
                <label asp-for="CategoriaId" class="form-label"></label>
            </div>
            <div class="form-floating mb-3">
                <input asp-for="Nombre" placeholder="Nombre" class="form-control" />
                <label asp-for="Nombre" class="form-label"></label>
                <span asp-validation-for="Nombre" class="text-danger"></span>
            </div>
        </div>
    </div>

    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
</form>
```



15. Dentro de la carpeta llamada `Views/Categorias` agregue un nuevo archivo llamado `Eliminar.cshtml` con el siguiente código.

```
@model Categoria
@{
    ViewData["Title"] = "Categorias";
    ViewData["SubTitle"] = "Eliminar";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

@if (!Model.Protegida)
{
    <p class="text-danger">¿Está seguro que desea eliminar este elemento?</p>
    <p class="text-danger">@ViewData["ErrorMessage"]</p>

    <partial name="_DetallePartial" />
    <form asp-action="Eliminar">
        <div class="text-center mt-3">
            <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Eliminar</button>
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </form>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            La categoría "<strong>@Model.Nombre</strong>" esta protegida y no puede eliminarse.
        </div>
    </div>

    <div class="text-center mt-3">
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
}
```

16. Dentro de la carpeta llamada **Views/Categorias** agregue un nuevo archivo llamado **Index.cshtml** con el siguiente código.

```
@model List<Categoria>
@{
    ViewData["Title"] = "Categorias";
    ViewData["SubTitle"] = "Listado";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<div class="row small mb-3">
    <div class="col">
        <a class="text-decoration-none" asp-action="Crear" title="Crear nuevo">Crear nueva</a>
    </div>
    <div class="col text-end">
        Mostrando @Model.Count() elementos
    </div>
</div>

@if (Model.Count() > 0)
{
    <div class="table-responsive">
        <table class="table table-striped table-bordered small">
            <thead class="text-center">
                <tr>
                    <th>
                        @Html.DisplayNameFor(model => model.First().CategoriaId)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Nombre)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Protegida)
                    </th>
                    <th></th>
                    <th></th>
                    <th></th>
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td>
                            @Html.DisplayFor(modelItem => item.CategoriaId)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Nombre)
                        </td>
                        <td>
                            @(item.Protegida ? "No" : "Sí")
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Detalle" asp-route-id="@item.CategoriaId">Detalle</a>
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Editar" asp-route-id="@item.CategoriaId">Editar</a>
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Eliminar" asp-route-id="@item.CategoriaId">Eliminar</a>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning role="alert">
            No se han encontrado elementos. Inténtelo de nuevo más tarde.
        </div>
    </div>
}
}
```

## Comprar

17. Dentro de la carpeta llamada `Views/Home` agregue un nuevo archivo llamado `_CardPartial.cshtml` con el siguiente código.

```
@model Producto

<div class="card h-100">
  <div class="card-header text-center bg-white pt-3">
    <img src='@((Model.ArchivoId == null) ? "https://placeholder.co/300x200/FFF/999?text=Articulo" : $"{ViewBag.Url}/api/archivos/{Model.ArchivoId}")'
      alt="@Model.Titulo" class="img-fluid" data-url="@ViewBag.Url"
      style="max-height: 200px; width: fit-content;">
    </div>
  <div class="card-body d-flex flex-column">
    <p class="card-text">@Model.Titulo</p>
    <h5 class="card-title">@Html.DisplayFor(model => model.Precio)</h5>
    <p class="card-text">
      @foreach (var cat in Model.Categorias!)
      {
        <span class="badge rounded-pill text-bg-warning">@cat.Nombre</span>
      }
    </p>
    <p class="mt-auto card-text text-success fw-semibold small"><i class="bi bi-lightning-fill"></i> Envío gratis</p>
  </div>
  <div class="card-footer">
    <a class="btn btn-primary btn-sm" href="#" role="button">Comprar</a>
  </div>
</div>
```

18. Dentro de la carpeta llamada `Views/Home` agregue un nuevo archivo llamado `Index.cshtml` con el siguiente código.

```
@model List<Producto>
@{
    ViewData["Title"] = "Comprar";
    ViewData["SubTitle"] = "Listado";
}
<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>
<div class="row small">
    <div class="col-12 col-sm-6 col-md-4 col-lg-3">
        <form asp-action="Index" method="get">
            <div class="input-group input-group-sm mb-3">
                <input name="s" id="s" value="@ViewBag.search" type="search" class="form-control"
                    placeholder="Buscar por título">
                <button class="btn btn-primary" type="submit" title="Buscar"><i class="bi bi-search"></i></button>
            </div>
        </form>
    </div>
</div>
<div class="row small mb-3">
    <div class="col text-end">
        Mostrando @Model.Count() elementos
    </div>
</div>
@if (Model.Count() > 0)
{
    <div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-4 g-4">
        @foreach (var item in Model)
        {
            <div class="col">
                <partial name="_CardPartial" model="item" />
            </div>
        }
    </div>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            No se han encontrado elementos. Inténtelo de nuevo más tarde.
        </div>
    </div>
}
```

## Home

19. Dentro de la carpeta llamada **Views/Home** agregue un nuevo archivo llamado **Index.cshtml** con el siguiente código.

```
@using System.Security.Claims
@{
    ViewData["Title"] = "Página principal";
}

<div style="left: 50%;margin-left:-50vw;position: relative;width: 100vw">
    <div id="carouselExample" class="carousel slide" style="margin-top: -1rem;">
        <div class="carousel-indicators">
            <button type="button" data-bs-target="#carouselExample" data-bs-slide-to="0" class="active"
                aria-current="true" aria-label="Slide 1"></button>
            <button type="button" data-bs-target="#carouselExample" data-bs-slide-to="1" aria-label="Slide 2"></button>
            <button type="button" data-bs-target="#carouselExample" data-bs-slide-to="2" aria-label="Slide 3"></button>
            <button type="button" data-bs-target="#carouselExample" data-bs-slide-to="3" aria-label="Slide 4"></button>
        </div>
        <div class="carousel-inner">
            <div class="carousel-item active">
                
            </div>
            <div class="carousel-item">
                
            </div>
            <div class="carousel-item">
                
            </div>
            <div class="carousel-item">
                
            </div>
        </div>
        <button class="carousel-control-prev" type="button" data-bs-target="#carouselExample" data-bs-slide="prev">
            <span class="carousel-control-prev-icon" aria-hidden="true"></span>
            <span class="visually-hidden">Previous</span>
        </button>
        <button class="carousel-control-next" type="button" data-bs-target="#carouselExample" data-bs-slide="next">
            <span class="carousel-control-next-icon" aria-hidden="true"></span>
            <span class="visually-hidden">Next</span>
        </button>
    </div>
</div>

<div class="container-fluid mt-4">
    <div class="card">
        <div class="card-header">
            Mercado Libre
        </div>
        <div class="card-body">
            <h5 class="card-title">Compre productos con Envío Gratis en Mercado Libre México.</h5>
            <p class="card-text">Encuentre miles de marcas y productos a precios increíbles.</p>
            <p class="card-text">Mercado Libre es la mayor plataforma de comercio electrónico de la región, donde millones de compradores y vendedores se encuentran para realizar transacciones con una amplia gama de productos y servicios, a precio fijo.</p>
            @if (User.Identity?.IsAuthenticated == true)
            {
                if (User.FindFirstValue(ClaimTypes.Role) == "Usuario")
                {
                    <a asp-controller="Comprar" class="btn btn-primary">Comenzar a comprar</a>
                }
                else if (User.FindFirstValue(ClaimTypes.Role) == "Administrador")
                {
                    <a asp-controller="Productos" class="btn btn-primary">Administrar catálogo</a>
                }
            }
            else
            {
                <a asp-controller="Auth" class="btn btn-primary">Iniciar sesión</a>
            }
        </div>
    </div>
</div>
```

## Productos

20. Dentro de la carpeta llamada `Views/Productos` agregue un nuevo archivo llamado `_CardPartial.cshtml` con el siguiente código.

```
@model Producto

<div class="card mt-3">
  <div class="row g-0">
    <div class="col-md-4 text-center bg-white pt-3">
      
    </div>
    <div class="col-md-8">
      <div class="card-body">
        <div class="mb-3 ps-3">
          <label asp-for="ProductoId" class="form-label form-text"></label>
          <div class="form-text text-body">@Model.ProductoId</div>
        </div>
        <div class="mb-3 ps-3">
          <label asp-for="Titulo" class="form-label form-text"></label>
          <div class="form-text text-body">@Model.Titulo</div>
        </div>
        <div class="mb-3 ps-3">
          <label asp-for="Precio" class="form-label form-text"></label>
          <div class="form-text text-body">@Model.Precio</div>
        </div>
        <div class="mb-3 ps-3">
          <label asp-for="ArchivoId" class="form-label form-text"></label>
          <div class="form-text text-body">@((Model.ArchivoId == null) ? "N/A" : $"Id: {Model.ArchivoId}")</div>
        </div>
        <div class="mb-3 ps-3">
          <label asp-for="Descripcion" class="form-label form-text"></label>
          <div class="form-text text-body">@Model.Descripcion</div>
        </div>
      </div>
    </div>
  </div>
</div>
```

21. Dentro de la carpeta llamada `Views/Productos` agregue un nuevo archivo llamado `CategoriasListadoPartial.cshtml` con el siguiente código.

```
@model List<Categoria>

<div class="row small mb-3">
  <div class="col">
    <a class="text-decoration-none" asp-action="CategoriasAgregar" asp-route-id="@ViewData["ProductoId"]"
      title="Agregar">Agregar</a>
  </div>
  <div class="col text-end">
    Mostrando @Model.Count() elementos
  </div>
</div>

@if (Model.Count() > 0)
{
  <div class="table-responsive">
    <table class="table table-striped table-bordered small">
      <thead class="text-center">
        <tr>
          <th>
            @Html.DisplayNameFor(model => model.First().CategoriaId)
          </th>
          <th>
            @Html.DisplayNameFor(model => model.First().Nombre)
          </th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        @foreach (var item in Model)
        {
          <tr>
            <td>
              @Html.DisplayFor(modelItem => item.CategoriaId)
            </td>
            <td>
              @Html.DisplayFor(modelItem => item.Nombre)
            </td>
            <td width="1">
              <a class="text-decoration-none small text-uppercase" asp-action="CategoriasRemover"
                asp-route-id="@ViewData["ProductoId"]" asp-route-categoriaid="@item.CategoriaId">Eliminar</a>
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>
}
else
{
  <div class="mt-5">
    <div class="alert alert-warning small" role="alert">
      No se han encontrado elementos.
    </div>
  </div>
}
```

22. Dentro de la carpeta llamada `Views/Productos` agregue un nuevo archivo llamado `Categorias.cshtml` con el siguiente código.

```
@model Producto
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Categorias";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<h4 class="my-3">Categorias</h4>
@if (Model.Categorias != null)
{
    <partial name="_CategoriasListadoPartial" model="Model.Categorias" />
}
<div class="text-center mt-3">
    <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
</div>

<h4 class="my-3">Producto</h4>
<partial name="_CardPartial" model="Model" />
```



23. Dentro de la carpeta llamada `Views/Productos` agregue un nuevo archivo llamado `CategoriasAgregar.cshtml` con el siguiente código.

```
@model ProductoCategoria
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Agregar categoría";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar a las categorías"
            asp-action="Categorias" asp-route-id="@Model.Producto?.ProductoId">Categorías</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<h4 class="my-3">Agregar categoría</h4>
<form asp-action="CategoriasAgregar" asp-route-id="@Model.Producto?.ProductoId">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="card-body small">
            <div class="form-floating mb-3">
                <select class="form-select" asp-for="CategoriaId" class="form-control" asp-items="ViewBag.Categoria">
                    <option value="">Seleccione una categoría</option>
                </select>
                <label asp-for="CategoriaId" class="control-label"></label>
                <span asp-validation-for="CategoriaId" class="text-danger" />
            </div>
        </div>
    </div>

    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Agregar</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Categorias" asp-route-id="@Model.Producto?.ProductoId"
            title="Cancelar">Cancelar</a>
    </div>
</form>

<h4 class="my-3">Producto</h4>
<partial name="_CardPartial" model="Model.Producto" />
```

24. Dentro de la carpeta llamada `Views/Productos` agregue un nuevo archivo llamado `CategoriasRemover.cshtml` con el siguiente código.

```
@model ProductoCategoria
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Remover categoría";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar a las categorías"
            asp-action="Categorias" asp-route-id="@Model.Producto?.ProductoId">Categorías</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<h4 class="my-3">Remover categoría</h4>
<p class="text-danger">¿Está seguro que desea remover este elemento?</p>
<p class="text-danger">@ViewData["ErrorMessage"]</p>

<form asp-action="CategoriasRemover" asp-route-id="@Model.Producto?.ProductoId">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="card-body small">
            <div class="form-floating mb-3">
                <div class="form-floating mb-3">
                    <input asp-for="CategoriaId" placeholder="CategoriaId" readonly class="form-control-plaintext" />
                    <label asp-for="CategoriaId" class="form-label"></label>
                </div>
                <div class="mb-3 ps-3">
                    <label asp-for="Nombre" class="form-label form-text"></label>
                    <div class="form-text text-body">@Model.Nombre</div>
                </div>
            </div>
        </div>
    </div>

    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Remover</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Categorias" asp-route-id="@Model.Producto?.ProductoId"
            title="Cancelar">Cancelar</a>
    </div>
</form>

<h4 class="my-3">Producto</h4>
<partial name="_CardPartial" model="Model.Producto" />
```

25. Dentro de la carpeta llamada **Views/Productos** agregue un nuevo archivo llamado **Crear.cshtml** con el siguiente código.

```
@model Producto
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Crear";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script src="~/js/producto_portada.js"></script>
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Crear">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="row g-0">
            <div class="col-md-4">
                
            </div>
            <div class="col-md-8">
                <div class="card-body small">
                    <div class="form-floating mb-3">
                        <input asp-for="Titulo" placeholder="Titulo" class="form-control" />
                        <label asp-for="Titulo" class="form-label"></label>
                        <span asp-validation-for="Titulo" class="text-danger"></span>
                    </div>
                    <div class="form-floating mb-3">
                        <input asp-for="Precio" placeholder="Precio" class="form-control" />
                        <label asp-for="Precio" class="form-label"></label>
                        <span asp-validation-for="Precio" class="text-danger"></span>
                    </div>
                    <div class="form-floating mb-3">
                        <select class="form-select" asp-for="ArchivoId" class="form-control"
                            asp-items="ViewBag.Archivo">
                            <option value="">Producto sin portada</option>
                        </select>
                        <label asp-for="ArchivoId" class="control-label"></label>
                        <span asp-validation-for="ArchivoId" class="text-danger" />
                    </div>
                    <div class="form-floating mb-3">
                        <textarea asp-for="Descripcion" placeholder="Poster" class="form-control form-control-sm"
                            style="height: 100px"></textarea>
                        <label asp-for="Descripcion" class="form-label"></label>
                        <span asp-validation-for="Descripcion" class="text-danger"></span>
                    </div>
                </div>
            </div>
        </div>
        <div class="text-center mt-3">
            <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </div>
</form>
```

26. Dentro de la carpeta llamada `Views/Productos` agregue un nuevo archivo llamado `Detalle.cshtml` con el siguiente código.

```
@model Producto
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Detalle";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<partial name="_CardPartial" model="Model" />
<div class="text-center mt-3">
    <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
</div>
```

27. Dentro de la carpeta llamada **Views/Productos** agregue un nuevo archivo llamado **Editar.cshtml** con el siguiente código.

```
@model Producto
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Editar";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script src="~/js/producto_portada.js"></script>
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Editar">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="row g-0">
            <div class="col-md-4 text-center bg-white pt-3">
                
            </div>
            <div class="col-md-8">
                <div class="card-body small">
                    <div class="form-floating mb-3">
                        <input asp-for="ProductoId" placeholder="Id" readonly class="form-control-plaintext" />
                        <label asp-for="ProductoId" class="form-label"></label>
                    </div>
                    <div class="form-floating mb-3">
                        <input asp-for="Titulo" placeholder="Nombre" class="form-control" />
                        <label asp-for="Titulo" class="form-label"></label>
                        <span asp-validation-for="Titulo" class="text-danger"></span>
                    </div>
                    <div class="form-floating mb-3">
                        <input asp-for="Precio" placeholder="Nombre" class="form-control" />
                        <label asp-for="Precio" class="form-label"></label>
                        <span asp-validation-for="Precio" class="text-danger"></span>
                    </div>
                    <div class="form-floating mb-3">
                        <select class="form-select" asp-for="ArchivoId" class="form-control"
                            asp-items="ViewBag.Archivo">
                            <option value="">Producto sin portada</option>
                        </select>
                        <label asp-for="ArchivoId" class="control-label"></label>
                        <span asp-validation-for="ArchivoId" class="text-danger" />
                    </div>
                    <div class="form-floating mb-3">
                        <textarea asp-for="Descripcion" placeholder="Poster" class="form-control form-control-sm"
                            style="height: 100px"></textarea>
                        <label asp-for="Descripcion" class="form-label"></label>
                        <span asp-validation-for="Descripcion" class="text-danger"></span>
                    </div>
                </div>
            </div>
        </div>
        <div class="text-center mt-3">
            <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </div>
</form>
```

28. Dentro de la carpeta llamada **Views/Productos** agregue un nuevo archivo llamado **Eliminar.cshtml** con el siguiente código.

```
@model Producto
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Eliminar";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<p class="text-danger">¿Está seguro que desea eliminar este elemento?</p>
<p class="text-danger">@ViewData["ErrorMessage"]</p>

<partial name="_CardPartial" model="Model" />
<form asp-action="Eliminar">
    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Eliminar</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
</form>
```

29. Dentro de la carpeta llamada **Views/Productos** agregue un nuevo archivo llamado **Index.cshtml** con el siguiente código.

```
@model List<Producto>
@{
    ViewData["Title"] = "Productos";
    ViewData["SubTitle"] = "Listado";
}
<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>
<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>
<div class="row small">
    <div class="col-12 col-sm-6 col-md-4 col-lg-3">
        <form asp-action="Index" method="get">
            <div class="input-group input-group-sm mb-3">
                <input name="s" id="s" value="@ViewBag.search" type="search" class="form-control"
                    placeholder="Buscar por título">
                <button class="btn btn-primary" type="submit" title="Buscar"><i class="bi bi-search"></i></button>
            </div>
        </form>
    </div>
</div>
<div class="row small mb-3">
    <div class="col">
        @if (ViewBag.SoloAdmin == true)
        {
            <a class="text-decoration-none" asp-action="Crear" title="Crear nuevo">Crear nueva</a>
        }
    </div>
    <div class="col text-end">
        Mostrando @Model.Count() elementos
    </div>
</div>
```

30. Debajo, agregue el despliegue de la tabla con elementos.

```
@if (Model.Count() > 0)
{
    <div class="table-responsive">
        <table class="table table-striped table-bordered small">
            <thead class="text-center">
                <tr>
                    <th width="1">
                        @Html.DisplayNameFor(model => model.First().ProductoId)
                    </th>
                    <th width="1">
                        @Html.DisplayNameFor(model => model.First().ArchivoId)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Titulo)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Precio)
                    </th>
                    <th></th>
                    @if (ViewBag.SoloAdmin == true)
                    {
                        <th></th>
                        <th></th>
                        <th></th>
                    }
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td>
                            @Html.DisplayFor(modelItem => item.ProductoId)
                        </td>
                        <td class="text-center" width="1">
                            <img src='@(item.ArchivoId == null) ? "https://via.placeholder.com/27x40" : $"{ViewBag.Url}/api/archivos/{item.ArchivoId}"'
                                alt="@item.Titulo" class="img-fluid img-thumbnail" style="max-height: 40px;" />
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Titulo)<br />
                            <span class="text-secondary-emphasis small d-none d-md-block">@Html.DisplayFor(modelItem =>
                                item.Descripcion)</span><br />
                            @foreach (var cat in item.Categorias!)
                            {
                                <span class="badge rounded-pill text-bg-secondary">@cat.Nombre</span>
                            }
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Precio)
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Detalle"
                                asp-route-id="@item.ProductoId">Detalle</a>
                        </td>
                        @if (ViewBag.SoloAdmin == true)
                        {
                            <td width="1">
                                <a class="text-decoration-none small text-uppercase" asp-action="Categorias"
                                    asp-route-id="@item.ProductoId">Categorias</a>
                            </td>
                            <td width="1">
                                <a class="text-decoration-none small text-uppercase" asp-action="Editar"
                                    asp-route-id="@item.ProductoId">Editar</a>
                            </td>
                            <td width="1">
                                <a class="text-decoration-none small text-uppercase" asp-action="Eliminar"
                                    asp-route-id="@item.ProductoId">Eliminar</a>
                            </td>
                        }
                    </tr>
                }
            </tbody>
        </table>
    </div>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning role="alert">
            No se han encontrado elementos. Inténtelo de nuevo más tarde.
        </div>
    </div>
}
```



## Perfil

31. Dentro de la carpeta llamada `Views/Perfil` agregue un nuevo archivo llamado `Index.cshtml` con el siguiente código.

```
@model AuthUser
@{
    ViewData["Title"] = "Perfil de usuario";
    ViewData["SubTitle"] = "Perfil";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"]</h2>

<div class="card mt-3">
    <div class="card-body">
        <div class="mb-3 ps-3">
            <label asp-for="Email" class="form-label form-text"></label>
            <div class="form-text text-body">@Model.Email</div>
        </div>
        <div class="mb-3 ps-3">
            <label asp-for="Nombre" class="form-label form-text"></label>
            <div class="form-text text-body">@Model.Nombre</div>
        </div>
        <div class="mb-3 ps-3">
            <label asp-for="Rol" class="form-label form-text"></label>
            <div class="form-text text-body">@Model.Rol</div>
        </div>
        <div class="mb-3 ps-3 small">
            <label asp-for="Jwt" class="form-label form-text"></label>
            <div class="form-text text-body">@Model.Jwt</div>
        </div>
        <div class="mb-3 ps-3">
            <label class="form-label form-text">Tiempo restante</label>
            <div class="form-text text-body">@ViewBag.timeRemaining</div>
        </div>
    </div>
</div>
```

## Usuarios

32. Dentro de la carpeta llamada `Views/Usuarios` agregue un nuevo archivo llamado `DetallePartial.cshtml` con el siguiente código.

```
@model Usuario

<div class="card mt-3">
  <div class="card-body">
    <div class="mb-3 ps-3">
      <label asp-for="Email" class="form-label form-text"></label>
      <div class="form-text text-body">@Model.Email</div>
    </div>
    <div class="mb-3 ps-3">
      <label asp-for="Nombre" class="form-label form-text"></label>
      <div class="form-text text-body">@Model.Nombre</div>
    </div>
    <div class="mb-3 ps-3">
      <label asp-for="Rol" class="form-label form-text"></label>
      <div class="form-text text-body">@Model.Rol</div>
    </div>
  </div>
</div>
```

33. Dentro de la carpeta llamada `Views/Usuarios` agregue un nuevo archivo llamado `Crear.cshtml` con el siguiente código.

```
@model UsuarioPwd
@{
    ViewData["Title"] = "Usuarios";
    ViewData["SubTitle"] = "Crear";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<form asp-action="Crear">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="card mt-3">
        <div class="card-body small">
            <div class="form-floating mb-3">
                <input asp-for="Email" class="form-control" placeholder="Email">
                <label asp-for="Email" class="form-label"></label>
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-floating mb-3">
                <input asp-for="Password" class="form-control" placeholder="Password">
                <label asp-for="Password" class="form-label"></label>
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-floating mb-3">
                <input asp-for="Nombre" placeholder="Nombre" class="form-control" />
                <label asp-for="Nombre" class="form-label"></label>
                <span asp-validation-for="Nombre" class="text-danger"></span>
            </div>
            <div class="form-floating mb-3">
                <select class="form-select" asp-for="Rol" class="form-control" asp-items="ViewBag.Rol">
                    <option value="">Seleccione un rol de usuario</option>
                </select>
                <label asp-for="Rol" class="control-label"></label>
                <span asp-validation-for="Rol" class="text-danger" />
            </div>
        </div>
    </div>

    <div class="text-center mt-3">
        <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
</form>
```

34. Dentro de la carpeta llamada `Views/Usuarios` agregue un nuevo archivo llamado `Detalle.cshtml` con el siguiente código.

```
@model Usuario
@{
    ViewData["Title"] = "Usuarios";
    ViewData["SubTitle"] = "Detalle";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

<partial name="_DetallePartial" />
<div class="text-center mt-3">
    <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
</div>
```

35. Dentro de la carpeta llamada **Views/Usuarios** agregue un nuevo archivo llamado **Editar.cshtml** con el siguiente código.

```
@model Usuario
@{
    ViewData["Title"] = "Usuarios";
    ViewData["SubTitle"] = "Editar";
}

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

@if (ViewBag.PuedeEditar)
{
    <form asp-action="Editar">
        <div asp-validation-summary="ModelOnly" class="text-danger"></div>
        <div class="card mt-3">
            <div class="card-body small">
                <div class="form-floating mb-3">
                    <input asp-for="Id" placeholder="Id" readonly class="form-control-plaintext" />
                    <label asp-for="Id" class="form-label"></label>
                </div>
                <div class="form-floating mb-3">
                    <input asp-for="Email" placeholder="Email" readonly class="form-control-plaintext" />
                    <label asp-for="Email" class="form-label"></label>
                </div>
                <div class="form-floating mb-3">
                    <input asp-for="Nombre" placeholder="Nombre" class="form-control" />
                    <label asp-for="Nombre" class="form-label"></label>
                    <span asp-validation-for="Nombre" class="text-danger"></span>
                </div>
                <div class="form-floating mb-3">
                    <select class="form-select" asp-for="Rol" class="form-control" asp-items="ViewBag.Rol">
                        <option value="">Seleccione un rol de usuario</option>
                    </select>
                    <label asp-for="Rol" class="control-label"></label>
                    <span asp-validation-for="Rol" class="text-danger" />
                </div>
            </div>
            <div class="text-center mt-3">
                <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Guardar</button>
                <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
            </div>
        </div>
    </form>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            El usuario actual "<strong>@Model.Email</strong>" no puede editarse.
        </div>
        <div class="text-center mt-3">
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </div>
}
```

36. Dentro de la carpeta llamada **Views/Usuarios** agregue un nuevo archivo llamado **Eliminar.cshtml** con el siguiente código.

```
@model Usuario
@{
    ViewData["Title"] = "Usuarios";
    ViewData["SubTitle"] = "Eliminar";
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item"><a class="text-decoration-none" title="Regresar al listado"
            asp-action="Index">Listado</a></li>
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>

<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>

@if (ViewBag.PuedeEditar)
{
    <p class="text-danger">¿Está seguro que desea eliminar este elemento?</p>

    <partial name="_DetallePartial" />
    <form asp-action="Eliminar">
        <div class="text-center mt-3">
            <button id="btnEnviar" type="submit" class="btn btn-sm btn-primary">Eliminar</button>
            <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
        </div>
    </form>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            El usuario actual "<strong>@Model.Email</strong>" no puede editarse.
        </div>
    </div>

    <div class="text-center mt-3">
        <a class="btn btn-sm btn-outline-secondary" asp-action="Index" title="Cancelar">Cancelar</a>
    </div>
}
```

37. Dentro de la carpeta llamada **Views/Usuarios** agregue un nuevo archivo llamado **Index.cshtml** con el siguiente código.

```
@model List<Usuario>
@{
    ViewData["Title"] = "Usuarios";
    ViewData["SubTitle"] = "Listado";
}
<nav aria-label="breadcrumb">
    <ol class="breadcrumb small">
        <li class="breadcrumb-item active" aria-current="page">@ViewData["SubTitle"]</li>
    </ol>
</nav>
<h2 class="text-center mb-3">@ViewData["Title"] <small class="text-muted fs-5">@ViewData["SubTitle"]</small></h2>
<div class="row small mb-3">
    <div class="col">
        <a class="text-decoration-none" asp-action="Crear" title="Crear nuevo">Crear nuevo</a>
    </div>
    <div class="col text-end">
        Mostrando @Model.Count() elementos
    </div>
</div>
```

38. Debajo, agregue el despliegue de la tabla con elementos.

```
@if (Model.Count() > 0)
{
    <div class="table-responsive">
        <table class="table table-striped table-bordered small">
            <thead class="text-center">
                <tr>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Id)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Email)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Nombre)
                    </th>
                    <th>
                        @Html.DisplayNameFor(model => model.First().Rol)
                    </th>
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td>
                            @Html.DisplayFor(modelItem => item.Id)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Email)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Nombre)
                        </td>
                        <td>
                            @Html.DisplayFor(modelItem => item.Rol)
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Detalle"
                                asp-route-id="@item.Email">Detalle</a>
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Editar"
                                asp-route-id="@item.Email">Editar</a>
                        </td>
                        <td width="1">
                            <a class="text-decoration-none small text-uppercase" asp-action="Eliminar"
                                asp-route-id="@item.Email">Eliminar</a>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
}
else
{
    <div class="mt-5">
        <div class="alert alert-warning" role="alert">
            No se han encontrado elementos. Inténtelo de nuevo más tarde.
        </div>
    </div>
}
```

39. Enhorabuena, ha realizado correctamente este apartado.



## Instrucciones. Configurar su programa principal

1. Abra su archivo `Program.cs` y agregue el siguiente código en la parte superior.

```
using frontendnet.Middlewares;
using frontendnet.Services;
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

// Agregamos los servicios
builder.Services.AddControllersWithViews();

// Soporte para consultar el API
var UrlWebAPI = builder.Configuration["UrlWebAPI"];
builder.Services.AddHttpContextAccessor();
builder.Services.AddTransient<EnviaBearerDelegatingHandler>();
builder.Services.AddTransient<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<AuthClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); });
builder.Services.AddHttpClient<CategoriasClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<UsuariosClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<RolesClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<ProductosClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<PerfilClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<ArchivosClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
builder.Services.AddHttpClient<BitacoraClientService>(httpClient => { httpClient.BaseAddress = new Uri(UrlWebAPI!); })
    .AddHttpMessageHandler<EnviaBearerDelegatingHandler>()
    .AddHttpMessageHandler<RefrescaTokenDelegatingHandler>();
```

2. Todo este bloque se encarga de registrar los servicios que la aplicación utilizará y construirla.

```
// Soporte para Cookie Auth
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.Cookie.Name = ".frontendnet";
        options.AccessDeniedPath = "/Home/AccessDenied";
        options.LoginPath = "/Auth";
        options.SlidingExpiration = true;
        options.ExpireTimeSpan = TimeSpan.FromMinutes(20);
    });

var app = builder.Build();
```

3. Observe como estamos configurando la autenticación por cookies de sesión.
4. A continuación, debajo agregue las configuraciones y funcionalidades de los servicios.

```
// Agregamos un middleware para el manejo de errores
app.UseExceptionHandler("/Home/Error");

app.UseStaticFiles();
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

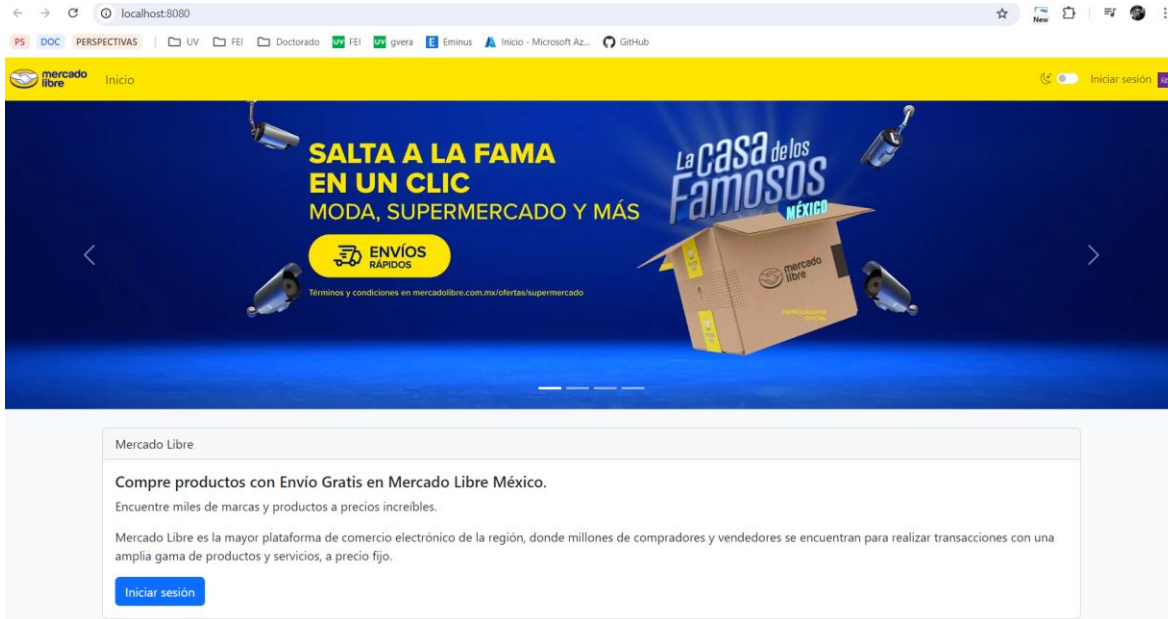
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

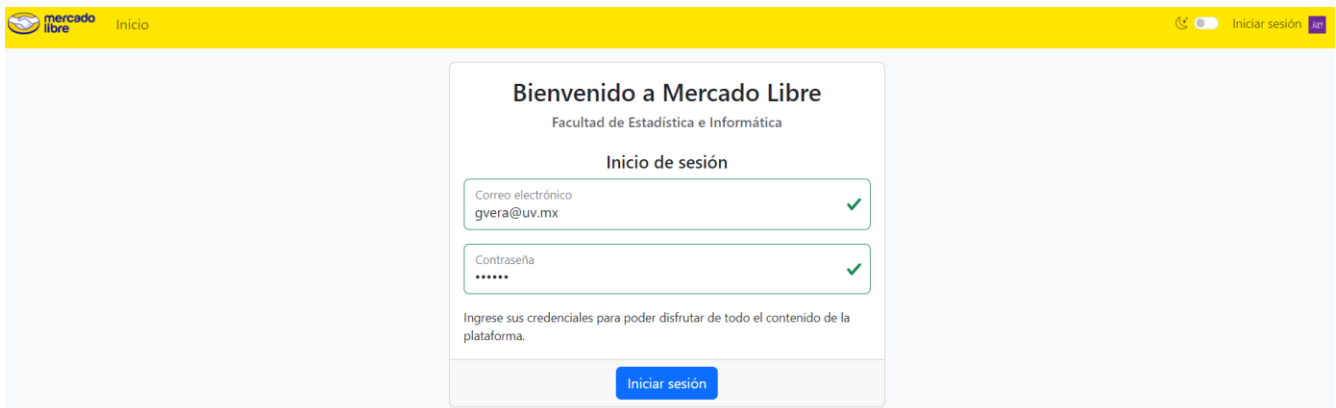
5. La última instrucción, ejecuta su aplicación.

## Probar el funcionamiento

1. Primero ejecute su proyecto de backend y manténgalo corriendo en el puerto **3000**.
2. Ejecute su aplicación frontend usando **dotnet run** o **dotnet watch run** y abra una ventana de su navegador en la dirección **http://localhost:8080/**.



3. Pruebe todos los botones de la parte pública y que todo funcione correctamente.
4. Inicie sesión con la cuenta de un usuario **administrador**.



5. Verifique que las validaciones funcionen correctamente.
6. Ahora pruebe todas las secciones del menú principal.



7. Agregue, modifique y elimine elementos de las secciones **Productos**, **Categorías**, **Usuarios**.

mercado libre Productos Categorías Archivos Usuarios Bitácora Hola gvera@uv.mx! Salir

Listado / Editar

### Productos Editar



Id  
1

Título  
Televisor Oled Smart Tv LG 42" 4k Uhd

Precio  
13080.00

Portada  
1722829802254-1.jpg

Descripción  
El Televisor Oled Smart Tv LG 42" 4k Uhd Tv Oled42c3psa 2023 es un producto de última tecnología que te permitirá disfrutar de una experiencia visual inigualable. Este modelo cuenta con una pantalla OLED de 42 pulgadas con resolución 4K UHD, lo que te permitirá ver tus contenidos favoritos con una calidad de imagen excepcional.

Guardar Cancelar

8. Revise que todas los apartados funcionen de manera adecuada incluyendo Archivos y Bitácora.

mercado libre Productos Categorías Archivos Usuarios Bitácora Hola gvera@uv.mx! Salir

Listado

### Archivos Listado

Agregar nuevo Mostrando 16 elementos

| Id | Poster                                                                              | Nombre              | MIME       | Tamprecio | Repositorio |         |        |          |
|----|-------------------------------------------------------------------------------------|---------------------|------------|-----------|-------------|---------|--------|----------|
| 1  |  | 1722829802254-1.jpg | image/jpeg | 38 KB     | Archivos    | DETALLE | EDITAR | ELIMINAR |
| 2  |  | 1722829807555-2.jpg | image/jpeg | 38 KB     | Archivos    | DETALLE | EDITAR | ELIMINAR |
| 3  |  | 1722829811242-3.jpg | image/jpeg | 14 KB     | Archivos    | DETALLE | EDITAR | ELIMINAR |
| 4  |  | 1722829816611-4.jpg | image/jpeg | 21 KB     | Archivos    | DETALLE | EDITAR | ELIMINAR |

9. Ahora ingrese con una cuenta de perfil usuario y observe los apartados Carrito y Comprar.

mercado libre Comprar Carrito Hola patito@uv.mx! Salir

### Comprar Listado

Buscar por título

Mostrando 15 elementos



10. Estos apartados se dejan abiertos para que el estudiante construya los artefactos correspondientes, al igual que Crear nueva cuenta para el auto registro de usuarios.

11. Enhorabuena, su aplicación funciona correctamente.

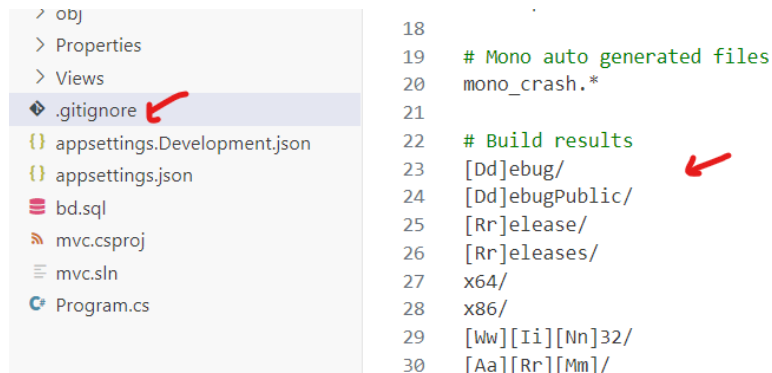
Instrucciones. Colocar su aplicación a un repositorio de código fuente.

1. Por último, antes de subir su aplicación a GitHub, genere un archivo llamado `.gitignore` que previene el subir información innecesaria al repositorio público.

```
dotnet new gitignore
```

```
PS C:\codigo\tw\mvc> dotnet new gitignore  
La plantilla "archivo gitignore de dotnet" se creó correctamente.
```

2. Este nuevo archivo aparece en la raíz de su proyecto.



3. Ahora ya puede publicar su archivo en GitHub para terminar la práctica.
4. Felicidades, ha creado su aplicación de manera correcta.